

СОДЕРЖАНИЕ

Термины и определения	5
Перечень сокращений и обозначений	6
Введение	7
1 Исследование предметной области	9
1.1 Обучение с подкреплением в условиях частичной наблюдаемости	9
1.1.1 Постановка задачи POMDP	9
1.1.2 RL-алгоритмы для визуальной навигации	9
1.1.3 ViZDoom как исследовательская платформа	10
1.2 Архитектуры памяти для обработки последовательностей	10
1.2.1 GRU — рекуррентный baseline	10
1.2.2 Трансформеры: внимание без затухания	11
1.2.3 Mamba-2: селективные SSM	13
1.2.4 Mamba-2 и Transformer: сравнение в задачах RL	15
1.2.5 Perceiver IO: отделение входа от вычислений	17
1.2.6 Сводная таблица архитектур	19
1.2.7 Обоснование выбора архитектур для сравнения	19
1.3 Sample Factory: асинхронный фреймворк для RL	19
1.3.1 Архитектура APPO	19
1.3.2 Интерфейс ModelCore и интеграция кастомных моделей	20
1.4 Постановка задачи исследования	20
2 Проектирование и разработка	21
2.1 Архитектура программного комплекса	21
2.2 Реализация Mamba2Core	22
2.2.1 Управление состоянием инференса	23
2.2.2 Интеграция с ModelCore sample-factory	24

2.2.3	Gradient checkpointing	24
2.2.4	Регистрация через фабрику	25
2.2.5	Параметры конфигурации	25
2.3	Реализация TransformerCore	25
2.4	Реализация PerceiverCore	26
2.5	Система НРО на базе Optuna	27
2.5.1	Байесовская оптимизация Mamba-2	27
2.5.2	НРО для Transformer	28
2.5.3	НРО для Perceiver IO	28
2.6	Экспериментальный стенд	28
3	Экспериментальное исследование	30
3.1	Методология сравнения	30
3.1.1	Метрики	30
3.1.2	Протокол	30
3.2	Пилотный скрининг архитектур	30
3.3	Сравнение GRU и Mamba-2	31
3.3.1	GRU: baseline и оптимизированный	31
3.3.2	Mamba-2: результаты НРО-конфигурации	31
3.3.3	GRU 250M: оценка потолка	32
3.3.4	Сводная таблица	32
3.4	Анализ результатов	33
3.4.1	Кривые обучения	33
3.4.2	Анализ потребления VRAM и скорости инференса	35
3.4.3	Обсуждение применимости архитектур	36
	Заключение	37
	Список использованных источников	39
	Приложение А Листинги кода	41

Приложение A.1 Mamba2StateEncoder	41
Приложение A.2 Mamba2Core.forward	42
Приложение A.3 Gradient checkpointing	43
Приложение A.4 Регистрация Mamba-2 в sample-factory	43
Приложение A.5 TransformerCore	44
Приложение A.6 PerceiverCore	46
Приложение Б Графики обучения	48
Приложение В Конфигурации экспериментов	51
Приложение В.1 Базовая конфигурация (APPO)	51
Приложение В.2 Конфигурация Mamba-2 (HPO best trial 62)	51
Приложение В.3 Конфигурация TransformerCore	52
Приложение В.4 Конфигурация PerceiverCore	52
Приложение В.5 Версии программного обеспечения	53
Приложение В.6 Потребление видеопамати (core-only)	53

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

- *Агент* — программа, взаимодействующая со средой, выбирающая действия на основе наблюдений и получающая награду.
- *Среда (окружение)* — внешняя система, с которой взаимодействует агент; в данной работе — ViZDoom.
- *Наблюдение (observation)* — входные данные агента на каждом шаге; в визуальном RL — кадр изображения.
- *Награда (reward)* — численный сигнал от среды, который агент стремится максимизировать.
- *Эпизод* — последовательность шагов агента от начала до терминального состояния (смерть, завершение уровня).
- *POMDP* — марковский процесс принятия решений с частичной наблюдаемостью.
- *Архитектура памяти* — модуль нейросети, обрабатывающий последовательности наблюдений и поддерживающий скрытое состояние.
- *Гиперпараметры (HP)* — параметры модели и алгоритма обучения, задаваемые до начала эксперимента.
- *Байесовская оптимизация (HPO)* — метод поиска гиперпараметров, строящий вероятностную модель целевой функции.
- *Gradient checkpointing* — техника уменьшения потребления видеопамяти за счёт пересчёта промежуточных активаций на обратном проходе.
- *Селективные SSM* — модели пространства состояний, где параметры динамики зависят от входного сигнала.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

- *APPO* – Asynchronous Proximal Policy Optimization
- *BPTT* – Backpropagation Through Time
- *FPS* – Frames Per Second
- *GPU* – Graphics Processing Unit
- *GRU* – Gated Recurrent Unit
- *HPO* – Hyperparameter Optimization
- *LSTM* – Long Short-Term Memory
- *MDP* – Markov Decision Process
- *POMDP* – Partially Observable Markov Decision Process
- *PPO* – Proximal Policy Optimization
- *RL* – Reinforcement Learning
- *SAC* – Soft Actor-Critic
- *SSM* – State Space Model
- *TPE* – Tree-structured Parzen Estimator
- *VRAM* – Video Random Access Memory

ВВЕДЕНИЕ

Агент действует в трёхмерном мире. Он видит только пиксели с камеры — ни координат, ни карты, ни полного состояния среды у него нет. Такая постановка называется частично наблюдаемым марковским процессом принятия решений (POMDP), и она стандартна для визуального RL. Данная проблема возникает во множестве прикладных областей: робототехника, беспилотные аппараты, игровые симуляторы — агент должен принимать решения по неполной информации.

ViZDoom — это API к DOOM (1993), адаптированный для машинного обучения [1]. Платформа предоставляет псевдотрёхмерное окружение с противниками, несколькими картами, динамическим освещением и — что существенно — со скоростью симуляции десятки тысяч кадров в секунду. За сутки на современном GPU можно набрать больше 800 миллионов наблюдений. Это делает ViZDoom эффективной платформой для экспериментов. В 2025-2026 годах опубликованы работы вроде NitroGen [2] и SIMA 2 [3], где агенты ориентируются в 3D-пространстве по картинке. Но вопрос выбора архитектуры памяти для RL-агента остаётся открытым. GRU [4], Transformer [5], Mamba-2 [6] и Perceiver IO [7] дают разный баланс между качеством политики, скоростью работы и потреблением памяти. Систематического сравнения всех четырёх в одинаковых условиях в литературе нет.

Понимание оптимальных архитектур памяти для визуального обучения с подкреплением создаёт основу для практических применений в робототехнике, автономной навигации и других областях, где агенты должны работать на основе визуальных входных данных в условиях частичной наблюдаемости. Практическая значимость состоит в том, что GRU-подобные архитектуры просты и надёжны (удобны для развёртывания), в то время как Mamba-2 предлагает решения с линейной сложностью для задач с длинным горизонтом планирования.

Целью работы является сравнительный анализ: взять единый алгоритм (APPO), единую среду (doom_benchmark) и одинаковый бюджет

обучения, после чего сопоставить архитектуры. Для этого требуется подключить ViZDoom к Sample Factory и организовать интерфейс смены модуля памяти, реализовать Mamba2Core, TransformerCore и PerceiverCore, провести пилотные запуски на 10 млн шагов для проверки сходимости, затем обучить каждую архитектуру на 50 млн шагов (с тремя seed для ключевых конфигураций) и сопоставить их по награде, FPS, пиковому VRAM и скорости, с которой награда выходит на плато.

Объект — нейросетевые архитектуры, обрабатывающие последовательности наблюдений; предмет — их применение в визуальной навигации при частичной наблюдаемости.

Новизна состоит в том, что четыре архитектуры ставятся в идентичные условия [8]: один и тот же алгоритм, одни и те же предобработка и бюджет, а не каждая со своими гиперпараметрами и окружением. Основное внимание уделено Mamba-2 — селективным SSM с линейной сложностью, которые в последнее время позиционируются как альтернатива трансформерам [9].

1 Исследование предметной области

1.1 Обучение с подкреплением в условиях частичной наблюдаемости

1.1.1 Постановка задачи POMDP

Обычный MDP: агент видит состояние s_t целиком, выбирает действие a_t , получает награду. Вся информация доступна.

В визуальных средах агент видит только картинку $o_t = O(s_t)$. По одному кадру невозможно восстановить положение агента в пространстве и состояние окружения. Это POMDP — частично наблюдаемый процесс. Чтобы принимать адекватные решения, агент должен помнить историю: $(o_1, a_1, \dots, o_t)$.

Без памяти политика $\pi(a_t \mid o_t)$ слепа к контексту. Агент реагирует на текущий кадр и не учитывает предшествующие наблюдения. Память накапливает наблюдения, сжимает их в скрытое состояние, обновляет на каждом шаге. Структура и принцип работы модуля памяти составляют основной вопрос исследования.

1.1.2 RL-алгоритмы для визуальной навигации

PPO [10] — стандарт для визуального RL. Surrogate loss клиппируется, политика не меняется слишком резко за один шаг. Это даёт стабильность. В работе используется APPO — асинхронная версия от Sample Factory [11].

SAC использует энтропийную регуляризацию. Политика поощряется за разведку, что хорошо для сред с разреженной наградой. Плата — replay buffer, который есть память. Для ViZDoom с его плотной наградой (убийства за шаг) SAC не даёт преимущества.

Decision Mamba — альтернативный подход, где SSM используется как сам алгоритм RL, а не как модуль памяти. Идея интересная, но на момент начала работы под ViZDoom не была апробирована. Рисковать не стали.

APPO выбран по трём причинам. Первая — асинхронный сбор опыта: workers не ждут learner, GPU загружен постоянно. Вторая — ModelCore: любую архитектуру памяти можно вставить, не трогая цикл обучения.

Третья — под `doom_benchmark` есть baseline от авторов фреймворка [11], с чем сравнивать результаты.

Таблица 1 — Сравнение RL-алгоритмов

Критерий	PPO	SAC	APPO
Тип	on-policy	off-policy	on-policy
Стабильность	Высокая	Средняя	Высокая
Параллелизм	Нет	Нет	Асинхронный
Буфер опыта	Нет	Replay buffer	Нет
VRAM	Умеренное	Высокое	Умеренное

1.1.3 ViZDoom как исследовательская платформа

ViZDoom — API к DOOM для ML [1]. Сценарии: лабиринты (`my_way_home`), бои (`defend_the_center`, `doom_benchmark`), сбор ресурсов (`health_gathering`). Каждый настраивается конфигурационным файлом.

Частичная наблюдаемость тут не искусственная, а естественная: вид от первого лица, узкий угол обзора. Никаких радаров. Ранние работы [12], [13] продемонстрировали, что DQN способен обучаться непосредственно с пикселей в DOOM, но качество политики напрямую зависело от того, как агент учитывал историю наблюдений.

В работе используется `doom_benchmark`. Во-первых, под него есть baseline от авторов Sample Factory [11]. Во-вторых, это стандартный сценарий для сравнения с литературой.

1.2 Архитектуры памяти для обработки последовательностей

1.2.1 GRU — рекуррентный baseline

GRU [4] представляет собой облегчённую версию LSTM [14]: два вентиля (обновления z_t и сброса r_t) вместо трёх, отсутствует отдельная ячейка памяти. При этом способность моделировать временные зависимости практически не уступает LSTM, а количество параметров на треть меньше.

Формально GRU описывается уравнениями:

$$r_t = \sigma(W_r \cdot [h_{\{t-1\}}, x_t]) \quad (1)$$

$$z_t = \sigma(W_z \cdot [h_{\{t-1\}}, x_t]) \quad (2)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{\{t-1\}}, x_t]) \quad (3)$$

$$h_t = (1 - z_t) * h_{\{t-1\}} + z_t * \tilde{h}_t \quad (4)$$

где h_t — скрытое состояние на шаге t , x_t — вход, r_t определяет, какую часть предыдущего состояния забыть, а z_t регулирует долю нового кандидата $\tilde{h}_t$.



Рисунок 1 — Структура GRU-ячейки: вход x_t и скрытое состояние $h_{\{t-1\}}$ подаются на вентили r_t и z_t , выход — взвешенная сумма старого состояния и кандидата.

В RL GRU прижился из-за предсказуемости: градиенты не взрываются, скрытое состояние 512 элементов занимает 2 КБ на среду, тысяча параллельных агентов не создаёт проблем с памятью. BPTT, recurrence=32. В Sample Factory GRU стоит по умолчанию [11], и он же выступает в качестве baseline.

1.2.2 Трансформеры: внимание без затухания

Трансформер [5] основан на механизме самовнимания (self-attention), который принципиально лишён проблемы затухающих градиентов — любые две позиции последовательности соединяются напрямую через вычисление весов внимания.

Механизм scaled dot-product attention определяется формулой:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \frac{K^\top}{\sqrt{d_k}}\right) V \quad (5)$$

где Q , K , V получаются линейным проецированием входной последовательности: $Q = XW_Q$, $K = XW_K$, $V = XW_V$. Деление на $\sqrt{d_k}$ предотвращает рост дисперсии скалярных произведений при большой размерности ключей.

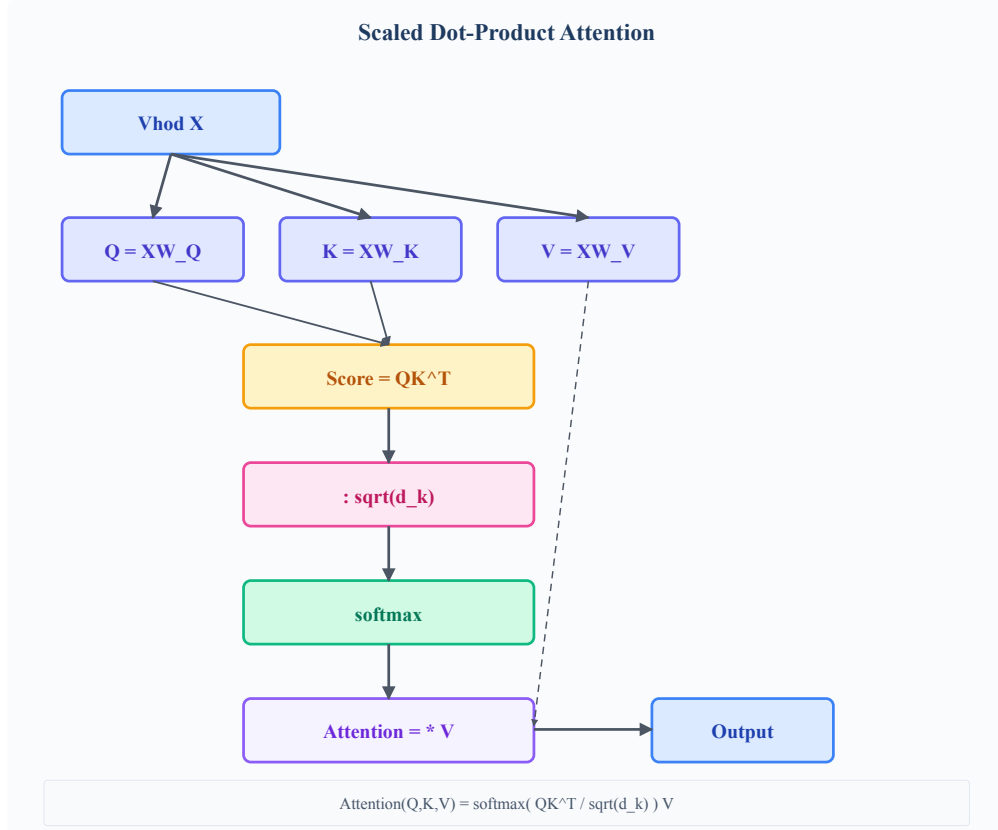


Рисунок 2 — Scaled dot-product attention: X проецируется в Q , K , V ; веса QK^T масштабируются, softmax, взвешенная сумма V .

Multi-head attention расширяет механизм, выполняя несколько операций внимания параллельно:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O \quad (6)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (7)$$

Каждая голова h_i работает в подпространстве пониженной размерности, что позволяет модели одновременно учитывать различные типы зависимостей между элементами последовательности.

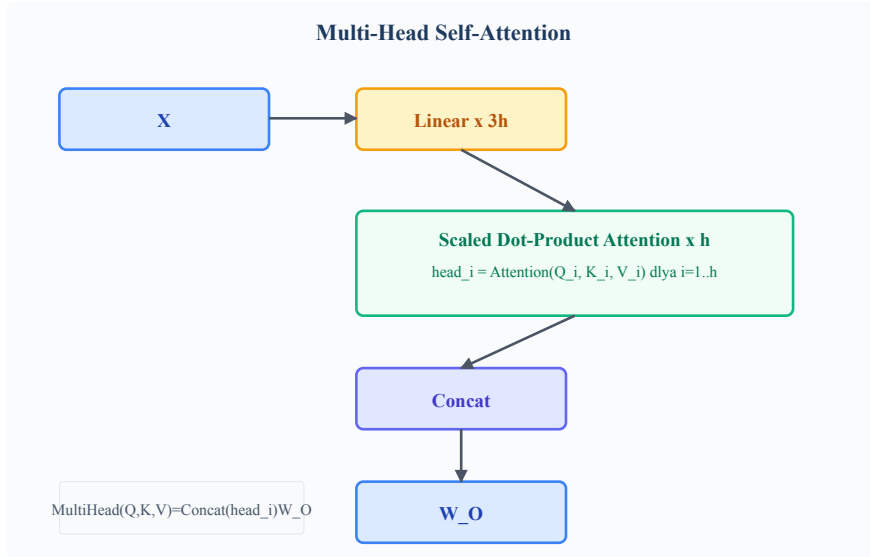


Рисунок 3 — Multi-head attention: h параллельных голов внимания, результаты конкатенируются и проецируются выходной матрицей W_O .

Цена самовнимания — $O(L^2)$ по длине последовательности: контекст больше 128 шагов уже не помещается в VRAM из-за необходимости хранить матрицу попарных весов размером $L \times L$. Если горизонт планирования короткий, выигрыша от внимания нет, хотя платить за квадратичную сложность всё равно приходится.

Трансформеры чувствительны к гиперпараметрам и требуют много данных; для `doom_benchmark` с окном 64 шага они, как показали эксперименты, не дают приемлемого качества. Причины этого разбираются в отдельном подразделе ниже.

1.2.3 Mamba-2: селективные SSM

Mamba-2 [6] основана на State Space Duality — по сути, линейная рекуррентная сеть, параметры которой зависят от входного сигнала. В отличие от классических рекуррентных сетей, где скрытое состояние обновляется по жёстко заданной схеме, Mamba-2 использует формализм пространства состояний (State Space Model, SSM).

SSM описывает систему парой дифференциальных уравнений:

$$\mathbf{h}'(t) = A\mathbf{h}(t) + Bx(t) \quad (8)$$

$$y(t) = C\mathbf{h}(t) + Dx(t) \quad (9)$$

где $\mathbf{h}(t) \in \mathbb{R}^n$ — вектор состояния размерности n , $x(t)$ — входной сигнал, $y(t)$ — выход. Матрица A определяет динамику системы (как состояние эволюционирует само по себе), B — как вход влияет на состояние, C — как состояние проецируется в выход, D — прямое пропускание входа на выход (skip-connection).

Для применения на дискретных последовательностях (шаги времени $t = 1, 2, \dots$) непрерывная система дискретизируется методом Zero-Order Hold (ZOH):

$$\mathbf{h}_t = \bar{A}\mathbf{h}_{\{t-1\}} + \bar{B}x_t \quad (10)$$

$$\bar{A} = \exp(\Delta A), \quad \bar{B} = (\Delta A)^{\{-1\}}(\exp(\Delta A) - I) \cdot \Delta B \quad (11)$$

Параметр Δ определяет шаг дискретизации — по сути, скорость обновления состояния.

Селективность. Ключевое отличие Mamba-2 от предшествующих SSM (S4, S5) — матрицы B , C и шаг Δ вычисляются от входного сигнала x_t , а не фиксируются после обучения:

$$B = f_B(x_t), \quad C = f_C(x_t), \quad \Delta = \text{softplus}(f_\Delta(x_t)) \quad (12)$$

где f_B , f_C , f_Δ — линейные проекции. Это делает SSM input-dependent, то есть модель сама решает, какую информацию запомнить, какую отбросить и с какой скоростью обновлять состояние — аналогично тому, как вентили GRU регулируют поток информации, но с более гибкой параметризацией.

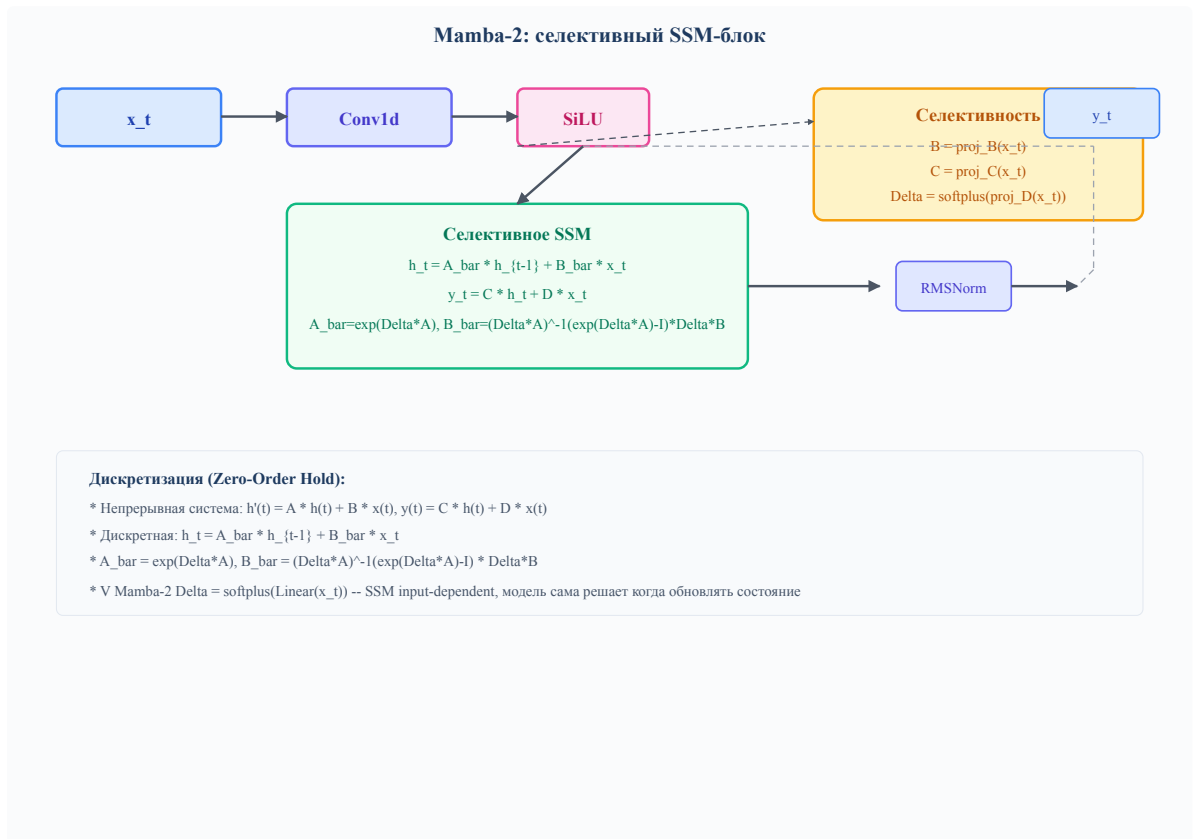


Рисунок 4 — Структура SSM-блока Mamba-2: вход $\rightarrow$ Conv1d + SiLU $\rightarrow$ селективный SSM (параметры B , C , Δ зависят от входа) $\rightarrow$ Residual + RMSNorm.

Архитектурно Mamba-2 состоит из: (1) дискретной свёртки (conv1d) с ядром 4, (2) нелинейности SiLU, (3) селективного SSM-ядра на Triton, (4) residual-соединения и RMSNorm. Сложность — $O(L)$, на длинных последовательностях Mamba-2 быстрее трансформера за счёт линейной рекурренты.

Предшественник Mamba [9] показал конкурентоспособные результаты на языке при линейном инференсе, но на ViZDoom систематически не проверялся. Отметим Spatial-Mamba [15] — адаптацию SSM для изображений, свидетельствующую, что селективные модели работают не только с текстом.

1.2.4 Mamba-2 и Transformer: сравнение в задачах RL

Результаты экспериментов (глава 3) показывают, что Transformer на doom_benchmark достигает лишь 1.01. Mamba-2 — 16.20. Разрыв огромный. Причины — фундаментальные, не в настройках.

Самовнимание считает попарные веса между всеми токенами окна. Для модели порядок не важен — его приходится добавлять через positional

encoding. В RL свежие наблюдения объективно важнее старых: кадр, поступивший только что, влияет на решение сильнее, чем кадр 60 шагов назад. Рекуррентные модели (GRU, Mamba-2) имеют это смещение «из коробки». Трансформер — нет.

Вторая проблема — квадратичная сложность. При окне 64 шага она ещё терпима. Но скрытое состояние трансформера нельзя «дополнить»: каждый новый кадр требует пересчёта всей матрицы внимания. Mamba-2 на инференсе просто обновляет состояние — $O(1)$ на шаг.

Третья — требовательность к данным. Трансформеры сходятся при объёмах, которые в RL труднодостижимы. 50M шагов в `doom_benchmark` — несколько дней счёта. Энкодер ViZDoom выдаёт 512-мерный признак. Для трансформера это мало, attention в таком пространстве плохо дифференцирует позиции.

Четвёртая — память. Трансформер хранит $L \times L$ матрицу весов: $64^2 = 4096$ на голову. При 8 головах и 4 слоях — 128K дополнительных элементов на шаг. Mamba-2 хранит `conv_state` ($4 \times 512 = 2048$) и `ssm_state` ($8 \times 128 \times 128 = 131072$). С ростом окна разница усугубляется.

Mamba-2 даёт компромисс: качество близко к GRU (16.20 против 17.86), $O(L)$ -инференс и константная память. Для визуального RL, где каждый шаг — миллисекунды, это преимущество. Трансформер при окне 64 шага не может развернуть механизм внимания и фактически не обучается.

Квадратичная сложность инференса. На каждом шаге трансформер пересчитывает внимание для всего окна контекста — $O(L^2)$. При окне 64 шага это ещё приемлемо само по себе, но в on-policy RL проблема в том, что скрытое состояние трансформера нельзя «дополнить»: при поступлении нового кадра необходимо пересчитать матрицу внимания целиком. Mamba-2, напротив, имеет рекуррентный инференс $O(1)$ на шаг после обработки последовательности — состояние фиксированного размера просто обновляется линейной операцией.

Отсутствие индуктивного смещения по времени. Self-attention вычисляет попарные веса между всеми позициями окна одновременно. Для

модели это означает, что порядок следования токенов не имеет значения — его приходится вносить искусственно через positional encoding. В RL же свежие наблюдения объективно важнее старых: кадр, поступивший только что, должен влиять на решение сильнее, чем кадр 60 шагов назад. Рекуррентные модели (GRU, Mamba-2) имеют это смещение «из коробки».

Требовательность к данным. Трансформеры сходятся при объёмах данных, которые в RL достижимы с трудом. On-policy методы генерируют опыт медленно: 50 М шагов в doom_benchmark — это несколько дней счёта на одном GPU. При этом энкодер ViZDoom (CNN) выдаёт 512-мерный признак — для трансформера это небольшая размерность, attention в таком пространстве плохо дифференцирует позиции.

Потребление памяти. Трансформер хранит матрицу $L \times L$ попарных весов ($64^2 = 4096$ элементов на голову). При 8 головах и 4 слоях — это дополнительно 128 тыс. элементов только для весов внимания на один шаг. Mamba-2 хранит только conv_state ($4 \times 512 = 2048$) и ssm_state ($8 \times 128 \times 128 = 131072$, при конкретной конфигурации HPO). Разница усугубляется с ростом окна контекста.

Mamba-2, таким образом, предлагает компромисс: качество, близкое к GRU (16.20 против 17.86 у оптимизированного GRU), при $O(L)$ -инференсе и константной памяти. Для задач визуального RL, где каждый шаг агента должен обрабатываться за миллисекунды, это существенное преимущество перед трансформером, который при окне 64 шага не может развернуть механизм внимания и фактически не обучается.

1.2.5 Perceiver IO: отделение входа от вычислений

Perceiver IO [7] (DeepMind) решает проблему зависимости вычислительной стоимости от длины входной последовательности принципиально иначе, чем SSM: модель оперирует фиксированным латентным массивом, размер которого не зависит от L .

Входная последовательность произвольной длины проецируется в K и V для кросс-внимания, где Q — это фиксированный набор обучаемых латентных векторов (latent array). После кросс-внимания латентный массив

обрабатывается стандартными трансформерными блоками (self-attention + FFN).

Формально, один шаг Perceiver:

$$Z_l = \text{LayerNorm}(\text{CrossAttn}(Z_{\{l-1\}}, X_{\{\epsilon\}})) \quad (13)$$

$$Z_{\{l+1\}} = \text{LayerNorm}(\text{TransformerBlock}(Z_l)) \quad (14)$$

где $Z \in \mathbb{R}^{M \times d}$ — латентный массив из M векторов (в данной работе $M = 32$), $X_{\{\epsilon\}}$ — спроецированный вход. Вычислительная стоимость: $O(ML)$ для кросс-внимания + $O(M^2)$ для самовнимания — линейна по L , причём коэффициент $M = 32$ мал.

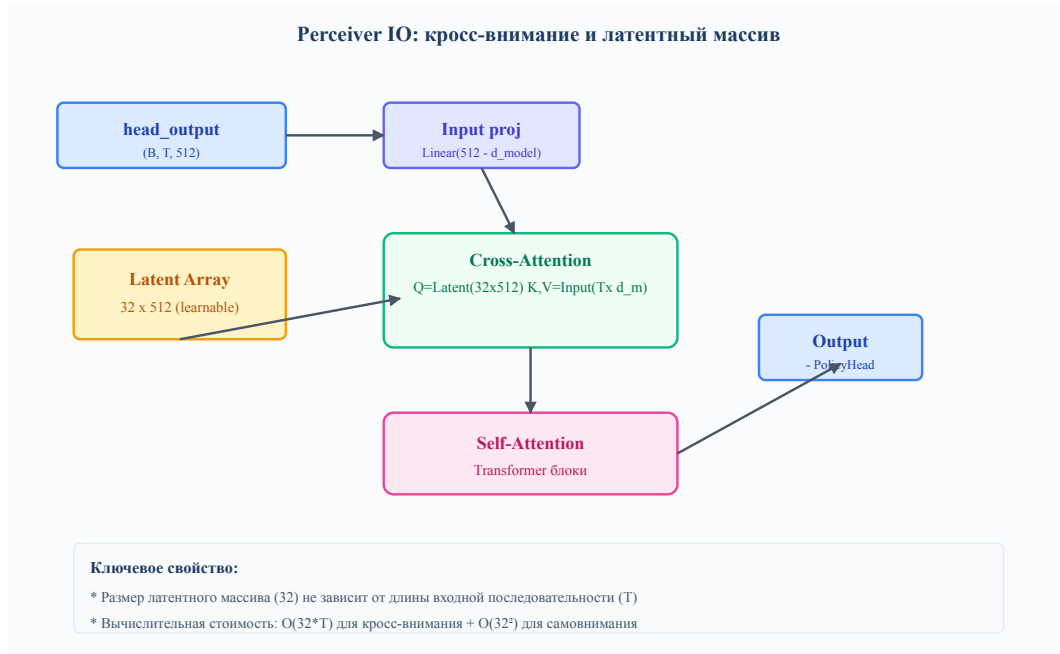


Рисунок 5 — Perceiver IO: head_output проецируется, через кросс-внимание сжимается в латентный массив (32 вектора), обрабатывается трансформерными блоками. Размер латентного массива не зависит от длины входа.

Для визуального RL это потенциально удобно — картинка 640x480 после энкодера даёт сотни временных признаков, и Perceiver обрабатывает их с постоянной стоимостью. На практике, однако, настройка оказалась сложной: размер массива, число итераций, способ инициализации латентного пространства — всё влияет на результат, и для doom_benchmark с фиксированным размером наблюдения преимущество переменного входа

не реализуется. Perceiver показал среднюю награду 1.97, что близко к случайной политике.

1.2.6 Сводная таблица архитектур

Таблица 2 — Сравнение архитектур памяти по сложности, размеру состояния, параметрам и типу инференса. $d = 512$, L — длина последовательности, $M = 32$.

Архитектура	Сложность	Скрытое состояние	Параметры (d=512)	Инференс
GRU	$O(L)$	d	$3d^2 \approx 0.8M$	Рекуррентный
Mamba-2	$O(L)$	$n_{\text{heads}} h d_{\text{state}}$ $d_{\text{conv}} d$	$\approx 2d^2$	Рекуррентный
Transformer	$O(L^2)$	$L \times d$	$4d^2 \approx 1.0M$ (4 слоя)	Пересчёт окна
Perceiver IO	$O(ML)$	$M \times d$	$\approx 2.5d^2$	Кросс-внимание

1.2.7 Обоснование выбора архитектур для сравнения

Из четырёх основных классов архитектур памяти выбрано по одному представителю: рекуррентные сети — GRU, внимание — Transformer, селективные SSM — Mamba-2, кросс-внимание — Perceiver IO. Все четыре сравнивались по трём критериям: качество обучения (mean reward после 50М шагов), скорость инференса (время одного forward-прохода) и потребление памяти (VRAM на обучение и инференс). Результаты сравнения представлены в главе 3.

1.3 Sample Factory: асинхронный фреймворк для RL

1.3.1 Архитектура APPO

Sample Factory [11] — RL-фреймворк, оптимизированный для высокой производительности. Policy workers собирают опыт параллельно и отправляют в очередь. Learner забирает данные и обновляет веса.

APPO: workers не ждут learner-а — работают с последней версией политики, какая есть. Learner накапливает траектории и обновляется, когда хватает данных. Схема обеспечивает высокую загрузку GPU: 8 workers ×

8 сред = 64 параллельных окружения, 10-20K FPS. Для ViZDoom такие показатели типичны.

1.3.2 Интерфейс ModelCore и интеграция кастомных моделей

ModelCore определяет два метода: `forward` принимает `head_output` и `rnn_states`, отдаёт новый выход и обновлённое состояние. `core_output_size` задаёт размерность для декодера. Любая реализация этого интерфейса подключается без модификации остального кода. `PackedSequence` используется для траекторий разной длины — градиенты не перетекают между эпизодами.

1.4 Постановка задачи исследования

Цель работы — сравнить GRU, Transformer, Mamba-2 и Perceiver IO в условиях ViZDoom по качеству, скорости и памяти при идентичных условиях обучения. Дополнительный вопрос — насколько Mamba-2 приближается к трансформеру по качеству, сохраняя линейную сложность инференса.

2 Проектирование и разработка

2.1 Архитектура программного комплекса

Архитектура строится по принципу разделения уровней: среда (ViZDoom), тренировочный цикл (Sample Factory / APPO) и модель политики (PolicyModel). Каждый уровень отвечает за свою функциональность и может быть заменён независимо.

Уровень среды. ViZDoom предоставляет API для взаимодействия с DOOM. Каждый экземпляр среды запускается в отдельном процессе и возвращает наблюдение (кадр 640x480, чёрно-белый), награду и флаг завершения эпизода. Sample Factory управляет пулом из $8 \times 8 = 64$ параллельных сред.

Уровень тренировочного цикла. Sample Factory реализует асинхронный APPO: workers собирают опыт, передают его на GPU для обновления политики, обновлённые веса рассылаются обратно. Асинхронность обеспечивает непрерывную загрузку GPU — узкое место не в вычислениях, а в пропускной способности конвейера данных.

Уровень модели политики (PolicyModel). Внутри модели данные проходят три этапа:

1. **CNNEncoder** — свёрточный энкодер ViZDoom обрабатывает стек из 4 последних кадров ($4 \times 640 \times 480$) через последовательность свёрточных слоёв с ReLU и flatten, выдавая признаковое представление размерности 512. На выходе энкодера формируется `head_output` — последовательность признаков длины T (batch размер), где T определяется рекурренцией (в данной работе 32) и типом обучения.
2. **ModelCore** — модуль памяти, заменяемый в зависимости от архитектуры. Получает `head_output` (PackedSequence на обучении, тензор на инференсе) и `rnn_states`, возвращает обновлённое скрытое состояние и выход. Может быть GRU, Mamba-2, Transformer или Perceiver IO.

3. **Decoder** — PolicyHead (линейный слой, выдающий распределение действий и значение состояния) и ValueHead (критик для APPO). Принимает выход ModelCore, возвращает логиты действий и скалярную оценку ценности состояния.

На рис. 6 приведена детальная архитектура PolicyModel: CNNEncoder, сменный модуль памяти (с четырьмя реализациями), и декодер. Пунктирной линией показан поток rnn_states, которым управляет Sample Factory.

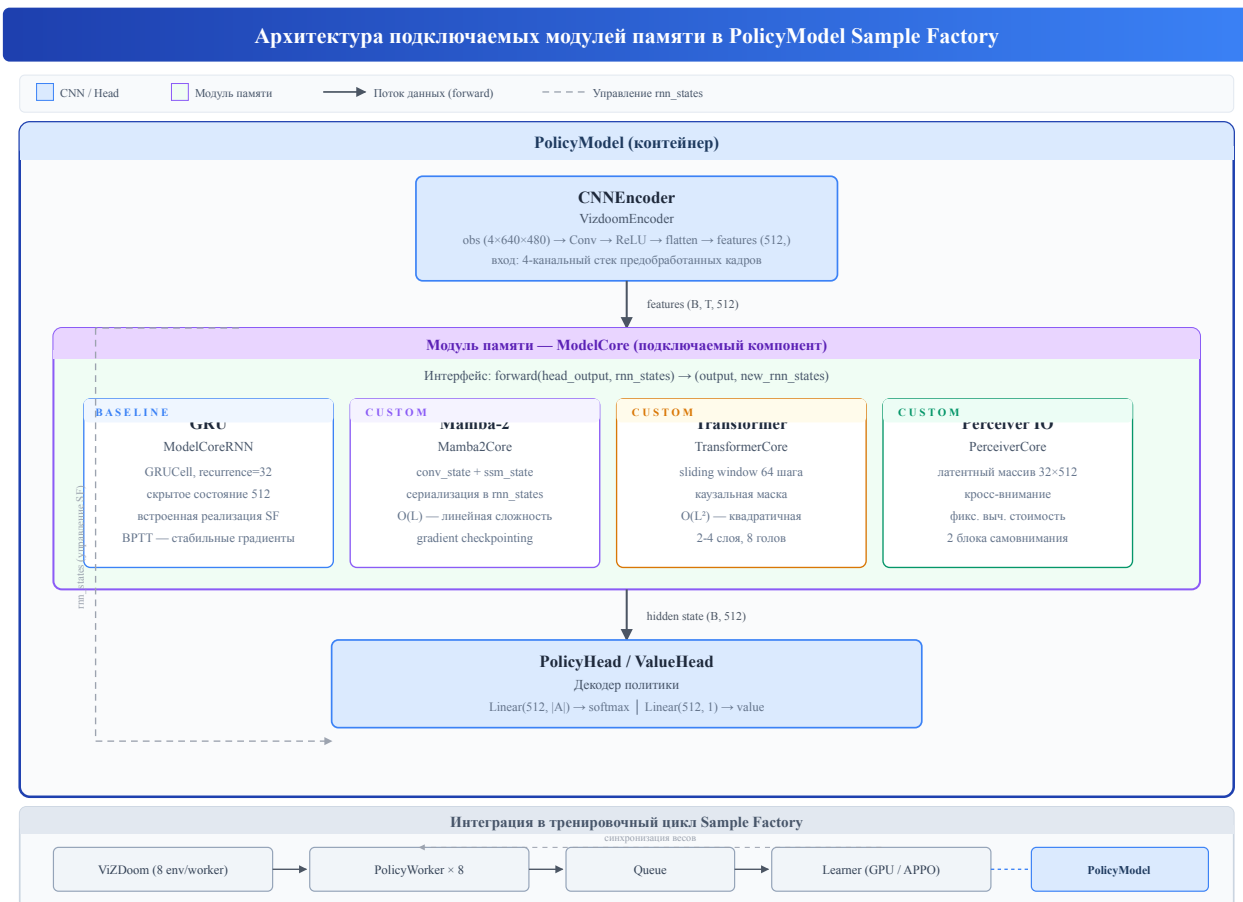


Рисунок 6 — Архитектура PolicyModel со сменным модулем памяти: GRU, Mamba-2, Transformer и Perceiver IO. CNNEncoder и PolicyHead/ValueHead фиксированы. Интеграция в тренировочный цикл Sample Factory.

2.2 Реализация Mamba2Core

Mamba-2 использует два внутренних состояния: conv_state и ssm_state. Они должны сохраняться между шагами эпизода и сбрасываться на его границе. Sample Factory поддерживает управление только одним массивом rnn_states, который обнуляется при завершении эпизода — оба состояния Mamba-2 нужно упаковать в один тензор.

2.2.1 Управление состоянием инференса

Ключевая задача при интеграции Mamba-2 в Sample Factory — упаковка двух внутренних состояний (`conv_state` и `ssm_state`) в единый массив `rnn_states` фиксированной длины, которым управляет фреймворк.

`Mamba2StateEncoder` (листинг B.1) сериализует `conv_state` ($d_{\text{conv}} \times d_{\text{model}} = 4 \times 512 = 2048$ элементов) и `ssm_state` ($n_{\text{heads}} \times h_{\text{headdim}} \times d_{\text{state}} = 8 \times 128 \times 128 = 131072$ элементов) в плоский тензор суммарной размерности, который совместим с `rnn_states`. На forward-проходе дешифратор выполняет обратную операцию: разделяет принятый тензор на две части и восстанавливает исходную размерность каждого состояния.

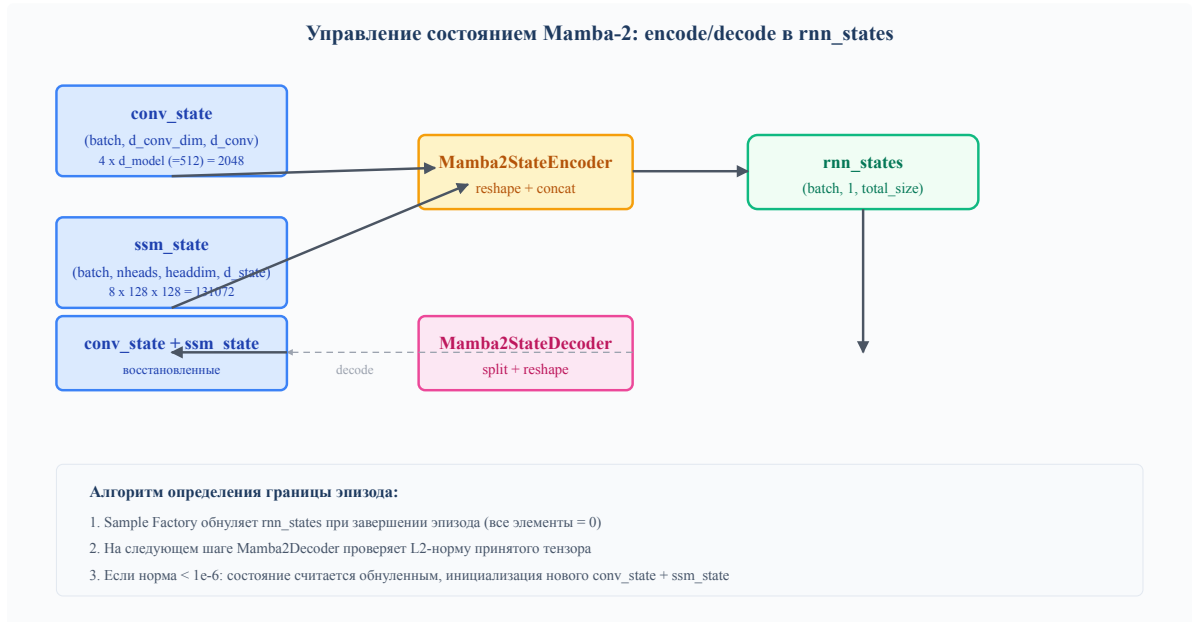


Рисунок 7 — Кодирование/декодирование состояния Mamba-2: `conv_state` и `ssm_state` сериализуются в `rnn_states`, затем декодируются. Граница эпизода — по L2-норме состояния.

Граница эпизода определяется по L2-норме принятого `rnn_states`: если норма меньше 10^{-6} , состояние считается обнулённым (Sample Factory сбросила его при завершении эпизода) и инициализируется заново. Порог, а не точный ноль, выбран из-за погрешностей при передаче чисел с плавающей точкой через очередь между процессами (`multiprocessing.Queue`).

Листинг B.1 — `Mamba2StateEncoder` (приведён в приложении B).

2.2.2 Интеграция с ModelCore sample-factory

Метод `forward` принимает `head_output` (`PackedSequence` на обучении, тензор на инференсе) и `rnn_states`, возвращает выход и новое состояние. На обучении `PackedSequence` распаковывается в 3D-тензор, проходит через Mamba-2 блоки, упаковывается обратно. На инференсе создается `InferenceParams`, состояние загружается из `rnn_states`, после прохода кодируется обратно.

Листинг B.2 — `Mamba2Core.forward` (приведён в приложении B).

2.2.3 Gradient checkpointing

При стандартном обучении PyTorch хранит все промежуточные активации каждого слоя для вычисления градиентов на обратном проходе. Для Mamba-2 с 4 слоями, скрытой размерностью 512 и батчем 4096 это даёт значительный расход VRAM: каждый слой хранит входной тензор (4096 x 512), выход после нормы, выход SSM-ядра и т.д.

Gradient checkpointing устраняет эту проблему: промежуточные активации не сохраняются, а пересчитываются заново на обратном проходе из предыдущего слоя. Это классический trade-off: вычисления заменяют память.

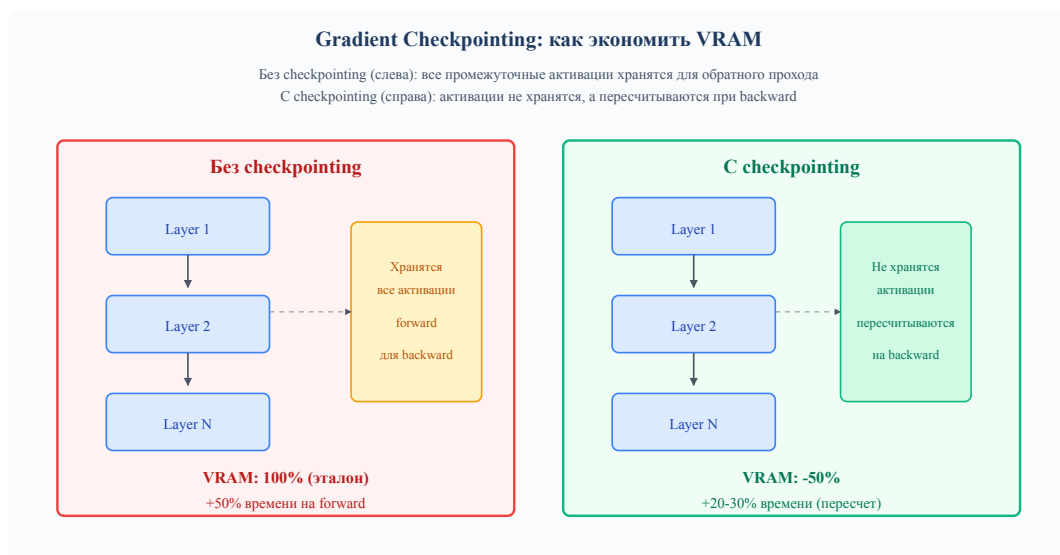


Рисунок 8 — Сравнение forward/backward без gradient checkpointing (слева) и с ним (справа). Без — активации хранятся в VRAM. С — пересчитываются, VRAM –50%, время +20–30%.

Снижение потребления VRAM составляет приблизительно 50% при увеличении времени вычислений на 20-30%. Включается параметром

`gradient_checkpointing = True`, каждый блок Mamba-2 обернут в `torch.utils.checkpoint.checkpoint` с флагом `use_reentrant=False`.

Листинг В.3 — Gradient checkpointing (приведён в приложении В).

2.2.4 Регистрация через фабрику

`Mamba2Factory` — обёртка, поддерживающая сериализацию `pickle`, которая регистрируется в глобальной фабрике моделей `sample-factory`. Это необходимо, так как `sample-factory` создаёт `workers` посредством `multiprocessing`, и фабрика должна сериализоваться.

Листинг В.4 — Регистрация Mamba-2 в `sample-factory` (приведён в приложении В).

После вызова `register_mamba2(cfg)` достаточно указать `rnn_type=„mamba2“` в конфиге — `sample-factory` сам создаст `Mamba2Core` при инициализации `workers`.

2.2.5 Параметры конфигурации

Модель настраивается группой `mamba`:

- `mamba_d_model = 512` — размерность скрытого пространства
- `mamba_d_state = 128` — размерность состояния SSM
- `mamba_d_conv = 4` — ядро свёртки
- `mamba_expand = 1` — расширение
- `mamba_headdim = 128` — размерность головы
- `mamba_ngroups = 1` — число групп

Значения — из НРО (раздел 2.5). Установлены равными 512, как у GRU, для честного сравнения.

2.3 Реализация `TransformerCore`

Стек трансформеров с каузальной маской [5]. 2-4 слоя, `pre-norm`, `dropout`, 512 скрытых, 8 голов. Окно контекста — 64 шага (увеличение ограничено доступным объёмом VRAM: матрица $64^2 = 4096$ коэффициентов на голову, при 8 головах и 4 слоях — 128К элементов только для внимания).

Особенность реализации в том, что трансформер не имеет рекуррентного скрытого состояния в классическом смысле. Для инференса

в on-policy RL используется механизм скользящего окна: последние 64 наблюдения сохраняются в буфере. Когда поступает новый кадр, буфер сдвигается (самый старый кадр удаляется), и трансформер обрабатывает всё окно заново, вычисляя полную матрицу внимания 64×64 .



Рисунок 9 — Скользящее окно TransformerCore на инференсе: при новом кадре окно сдвигается, старый токен вытесняется. Полный пересчёт $O(L^2)$ внимания на каждом шаге.

Данный подход требует большего объёма вычислений по сравнению с Mamba-2 ($O(L^2)$ против $O(L)$ за шаг), но даёт доступ ко всей истории внутри окна без сжатия в вектор фиксированной размерности.

Листинг B.5 — TransformerCore (приведён в приложении B).

2.4 Реализация PerceiverCore

Вход проецируется на латентный массив через кросс-внимание [7]. Массив обрабатывается трансформерными блоками. Размер массива настраивается. Начальная конфигурация: 32 латентных вектора размерности 512.

Схема работы: латентный массив (32) — Q , спроецированный вход (длина T) — K и V . После кросс-внимания размерность падает с T до 32. Дальше — только самовнимание и FFN над сжатым представлением.

На инференсе массив кодируется в `rnn_states`. Размер состояния Perceiver фиксирован — не растёт с длиной окна, в отличие от трансформера.

Листинг B.6 — PerceiverCore (приведён в приложении B).

2.5 Система НРО на базе Optuna

2.5.1 Байесовская оптимизация Mamba-2

Optuna, TPE-семплер (Tree Parzen Estimator), ASHA-прунинг после 10 эпох. Пространство поиска: d_{model} 256/512/1024, d_{state} 64/128, headdim 64/128, lr $1\text{e-}5..1\text{e-}3$, weight_decay 0..0.1.



Рисунок 10 — НРО pipeline: Optuna (TPE-семплер) → trial на 50М шагов, ASHA-прунинг после 10 эпох. Из 85 trials завершились 37 (54% прерываний).

Проведено 85 trials по 50М шагов, из которых 37 успешно завершились (54% прерываний из-за OOM или timeout).

Лучший trial 62: $d_{\text{model}}=512$, $d_{\text{state}}=128$, headdim=128, lr= $4.05\text{e-}4$, weight_decay=0.00196, expand=1. Mean reward 13.34, within-run std 3.14, max 16.94. Оптимизация заняла 42 минуты (2507 секунд на GPU). Все лучшие trial используют AdamW с lr $4\text{e-}4$ и weight_decay 0.002.

Таблица 3 — Пространство поиска НРО и результаты лучшего trial 62 — Mamba-2

Параметр	Диапазон поиска	Лучшее значение	Влияние
d_{model}	256 / 512 / 1024	512	Высокое
d_{state}	64 / 128	128	Среднее
headdim	64 / 128	128	Низкое
lr	$10^{-5}..10^{-3}$	4.05×10^{-4}	Высокое
weight_decay	0..0.1	1.96×10^{-3}	Среднее
expand	1 / 2	1	Низкое

2.5.2 НРО для Transformer

Проведено 15 trials для TransformerCore (то же пространство, добавлены window_size 32/64/128, dim_feedforward 1024/2048, dropout 0.1/0.2). Все 15 успешно завершены — Transformer стабилен и не требует больших ресурсов.

Лучший trial 5: best_reward=2.03 (d_model=512, nhead=8, window_size=64, dim_feedforward=1024, dropout=0.2, lr=8.36e-4, AdamW). Прирост относительно конфигурации по умолчанию (1.01) — всего 2×. Это подтверждает, что окно контекста 64 шага — фундаментальное ограничение: трансформер не видит дальше этого горизонта, а для doom_benchmark необходимо помнить существенно более длинные зависимости. Увеличение window_size до 128-256 — предмет дальнейших экспериментов.

2.5.3 НРО для Perceiver IO

10 trials по 25М шагов, 49 минут на trial. На момент написания завершён trial 0: best_reward=1.67 (d_model=512, latents=16, d_latents=256, blocks=1, heads=8, lr=3.9e-4, AdamW). Это ниже конфигурации по умолчанию (1.97).

Сложность настройки Perceiver IO выше, чем у Mamba-2: помимо d_model, lr, weight_decay надо подбирать размер латентного массива (16/32/64), размерность латентного пространства (256/512), число блоков (1/2/4), число голов (4/8). Пространство поиска — комбинаторно большое. Результаты раньше 6-8 trials ждать не стоит.

2.6 Экспериментальный стенд

Все эксперименты — на одном ноутбуке с RTX 5090 Laptop GPU. Характеристики — в табл. 4.

Таблица 4 — Характеристики экспериментального стенда

Компонент	Характеристика	Значение
CPU	Процессор	Intel Ultra 9 285H (16 ядер, 22 потока)
GPU	Видеокарта	NVIDIA RTX 5090 Laptop GPU (24 GB VRAM)

Компонент	Характеристика	Значение
RAM	Оперативная память	32 GB DDR5-5600
OC	Операционная система	Linux (Ubuntu 24.04)
CUDA	Версия CUDA	13.0
cuDNN	Версия cuDNN	9.1.9
Python	Интерпретатор	3.13.12
PyTorch	Фреймворк	2.11.0+cu130
SF	Sample Factory	2.1.1
ViZDoom	Среда	1.3.0

Параметры обучения — фиксированы для всех архитектур:

- APPO, doom_benchmark, 8 workers × 8 сред = 64 параллельных среды
- Batch 4096, rollout 64, recurrence 32, hidden 512
- Наблюдения: ч/б 640x480, FRAME_SKIP=4, стек 4 кадров
- Награда: дельта убийств за шаг (стандартная для doom_benchmark)
- Seed по умолчанию 0. Для seed=42 — аргумент командной строки (в конфигах запусков зафиксировано)
- config.json в train_dir для каждого запуска — для воспроизводимости
- FPS — утилитой sample-factory, среднее за всё обучение
- Метрики в metrics.json каждые 1000 шагов, плюс wandb через callback

Каждый seed запускался в отдельной tmux-сессии. Конфигурация каждого запуска — в train_dir/config.json. Всего около 150 запусков: 85 trials HPO Mamba-2, 15 trials Transformer, 10 trials Perceiver, плюс основной эксперимент с 3-4 seed для ключевых конфигураций. Суммарное GPU-время — около 600 часов.

3 Экспериментальное исследование

3.1 Методология сравнения

3.1.1 Метрики

Основная метрика — средняя награда за эпизод (mean reward).

Дополнительно:

- **Best reward** — максимальная средняя награда за всё обучение.
- **Within-run std** — стандартное отклонение внутри одного прогона (стабильность политики).
- **Across-seed std** — разброс между seed одной архитектуры (воспроизводимость).
- **FPS** — шагов среды в секунду.
- **VRAM** — пиковое потребление видеопамяти.

3.1.2 Протокол

Каждая архитектура — 50М шагов. Этапы:

1. **Запуск.** sample-factory с фиксированной конфигурацией. Чекпоинт каждые 10М шагов.
2. **Оценка.** Выбирается чекпоинт с максимальной средней наградой (best reward). Утилита sample-factory считает среднюю за последние 100 эпизодов.
3. **Повторение.** GRU (baseline и оптимизированный) и Mamba-2 — по 3-4 seed. Остальным — по одному (при низкой награде разброс несущественен).
4. **Длительный прогон.** GRU на 250М шагов — чтобы оценить потолок рекуррентной архитектуры.

3.2 Пилотный скрининг архитектур

Первый этап — быстрая отбраковка. Пять архитектур, 50М шагов, один seed. Результаты — на рис. 11 и в табл. 5.

GRU с гиперпараметрами от Mamba-2 — 20.15. Mamba-2 — 19.10 (лучший seed). Стандартный GRU — 13.58. Mamba-1 отстала — 12.59. Transformer и Perceiver IO неконкурентоспособны: 1.01 и 1.97.

Причины по архитектурам. Трансформеру не хватает окна контекста: 64 токена для ViZDoom — крайне мало. HPO поднял до 2.03, но порядок не

изменился. Perceiver IO теряет информацию в кросс-внимании, а его козырь (переменный вход) в ViZDoom не нужен. Mamba-1 уступает Mamba-2 из-за меньшего `d_state` (16 против 128) и неселективного SSM.

Дальше сравниваем только GRU и Mamba-2. Три архитектуры отсеяны по фундаментальным причинам, не связанным с подбором гиперпараметров.

3.3 Сравнение GRU и Mamba-2

3.3.1 GRU: baseline и оптимизированный

GRU со стандартными гиперпараметрами Sample Factory показал по трём seed: 12.43, 14.37 и 13.95 — средняя лучшая награда 13.58 ± 1.02 . Разброс между seed порядка 15% типичен для on-policy алгоритмов; within-run std по последним 20% итераций составил 0.33–0.70. FPS варьируется от 24 000 до 56 000 (зависит от загрузки системы), GPU загружен на 65%.

Когда к GRU применили гиперпараметры, найденные НПО для Mamba-2 (`AdamW`, `lr=4.05e-4`, `weight_decay=0.00196`, `exploration_loss_coeff=0.002`), результаты существенно изменились: seed1 — 17.44, seed2 — 16.00, seed3 — 20.15, средняя — 17.86 ± 2.11 . Прирост относительно baseline — +31%, причём seed3 (20.15) приближается к результату GRU 250M (22.45) при пятикратной экономии вычислительных ресурсов. Стандартные гиперпараметры Sample Factory для `doom_benchmark`, оказываются далеки от оптимальных — настройка важна не меньше, чем выбор архитектуры.

3.3.2 Mamba-2: результаты НПО-конфигурации

Mamba-2 с конфигурацией `trial 62` (`d_model=512`, `d_state=128`, `headdim=128`, `lr=4.05e-4`) запускали на 4 seed. Результаты: 14.87, 14.84, 19.10, 15.97 — средняя 16.20 ± 2.01 , максимальная 19.10. Высокий across-seed std (2.01) и within-run std (0.61–1.28) — следствие природы среды: награда в `doom_benchmark` скачет от эпизода к эпизоду в зависимости от поведения противника, и это не дефект архитектуры. По сходимости Mamba-2 стабильна, но к гиперпараметрам чувствительна: без НПО качество падает на 20–30%.

3.3.3 GRU 250M: оценка потолка

Длительный эксперимент с GRU на 250M шагов подтвердил плато: best_reward 22.45, финальная награда 20.95, средняя по последним 20% итераций — 20.28 ± 0.66 . После 150M шагов рост прекращается; FPS 56 455 — снижения скорости нет.

Потребление видеопамяти core-модулями замерялось отдельно. Результаты приведены в табл. В.6. Полная же модель, как видно из сводной таблицы далее, потребляет от 5 до 9 GB: энкодер занимает основной объём.

3.3.4 Сводная таблица

Таблица 5 — Сводные результаты экспериментов. ^aСредняя лучшая награда. ^bAcross-seed std. ^cПосле HPO. ^dGRU 250M: по последним 20% итераций.

Архитектура	Seed	Mean ^a	Std ^b	Max	FPS	VRAM
GRU baseline	3	13.58	1.02	14.37	24–56K	6 GB
GRU + Opt.HP	3	17.86	2.11	20.15	17–25K	6 GB
Mamba-2 HPO	4	16.20	2.01	19.10	15–23K	7 GB
Mamba-1	1	12.59	—	12.59	37K	6 GB
Transformer	1	1.01	—	2.03 ^c	29K	6 GB
Perceiver IO	1	1.97	—	1.97	8K	8 GB
GRU 250M	1	20.28 ^d	0.66 ^d	22.45	56K	6 GB

3.4 Анализ результатов

3.4.1 Кривые обучения

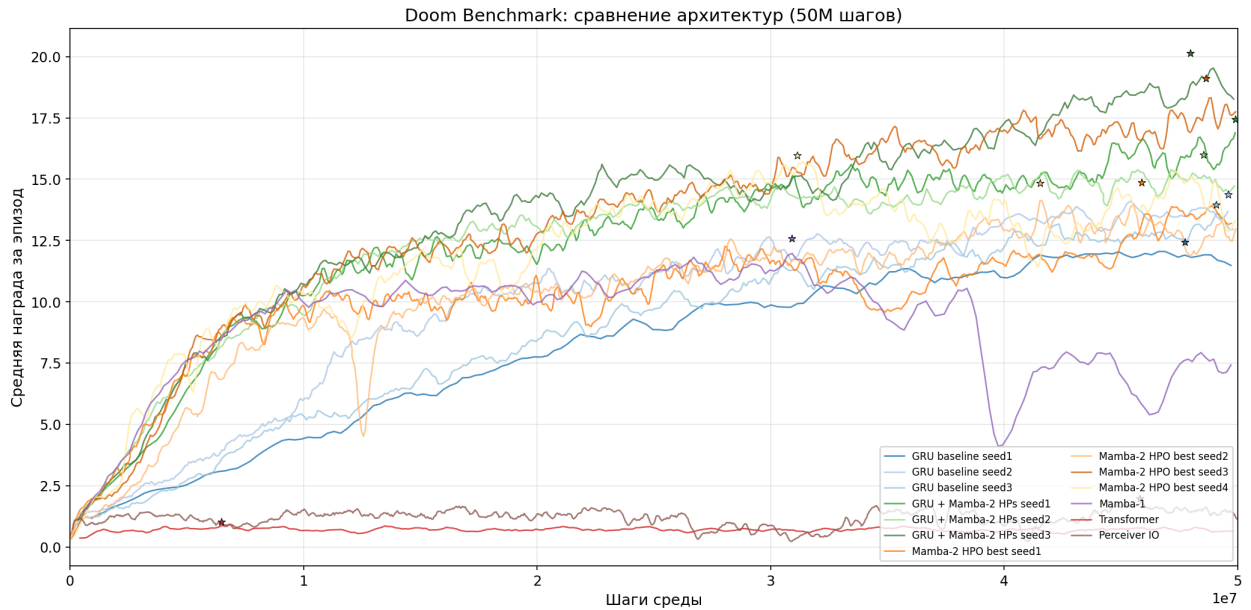


Рисунок 11 — Кривые обучения всех архитектур на 50М шагов. Для архитектур с несколькими seed показан лучший. Mamba-1, Transformer и Perceiver IO уступают GRU и Mamba-2.

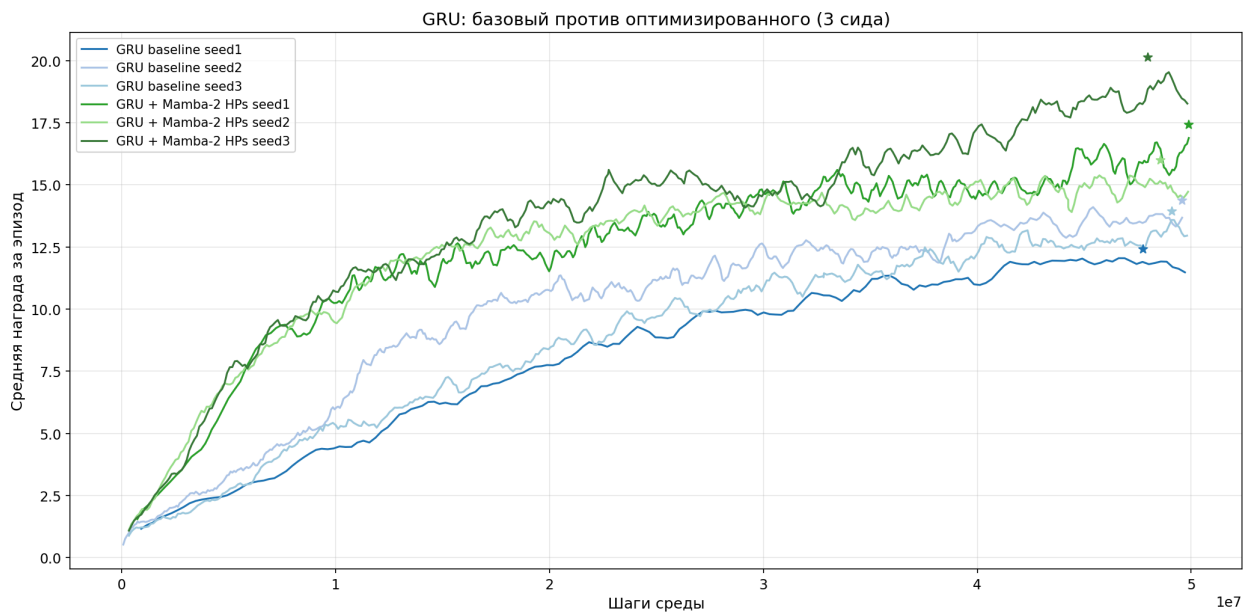


Рисунок 12 — GRU baseline vs GRU с гиперпараметрами Mamba-2 (3 seed). Оптимизированная GRU опережает baseline на всех seed (max 20.15).

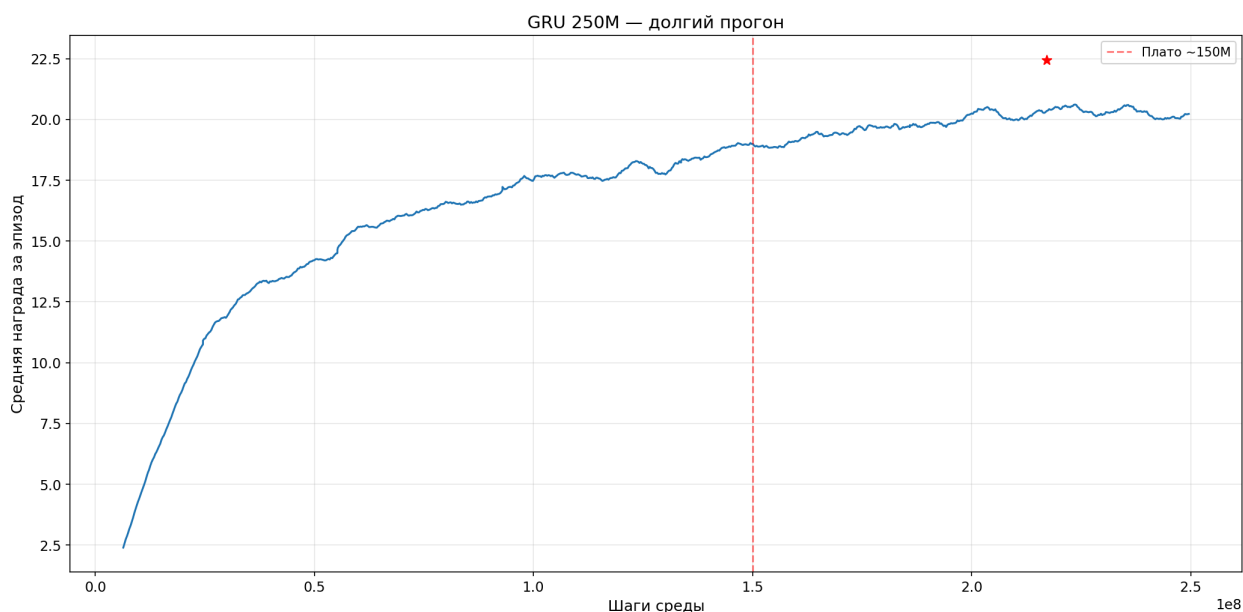


Рисунок 13 — GRU 250M — долгий прогон. Плато после 150M шагов, финальная награда 20.95, максимальная 22.45

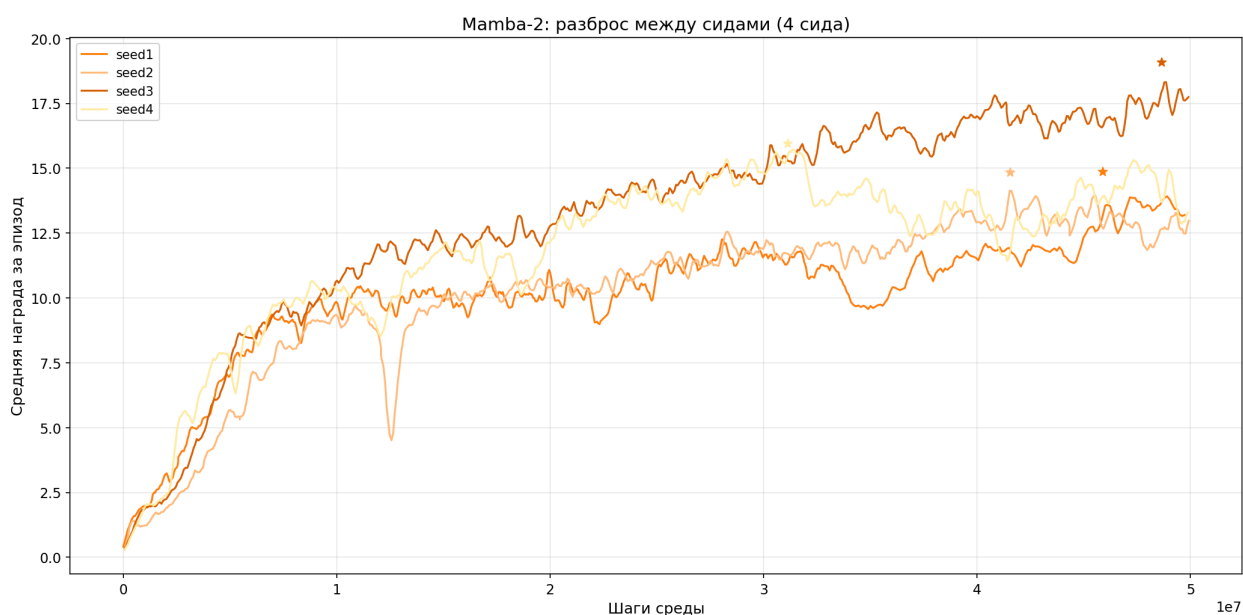


Рисунок 14 — Mamba-2 HPO: разброс между 4 seed. Seed3 показывает 19.10, остальные — 14.8–16.0. Высокая вариативность характерна для on-policy RL

GRU с гиперпараметрами Mamba-2 выходит на 55% финальной награды за первые 10M шагов (рис. 11). Mamba-2 обучается плавнее, но разброс между seed выше. GRU 250M выходит на плато около 150M (рис. 13). Mamba-1, Transformer и Perceiver IO не поднимаются выше 12-13 за 50M.

Характер обучения разный. GRU — резкий рост, потом плато. Mamba-2 — плавно, без скачков. Возможно, селективному SSM нужно больше данных, чтобы настроить входозависимые параметры.

Разрыв между GRU (17.86) и Mamba-2 (16.20) — менее 10%. Across-seed std — 2.01 и 2.11 соответственно. На обе архитектуры влияет стохастичность on-policy RL: сходимость чувствительна к начальному seed.

Ранжирование: GRU opt (17.86 ± 2.11), Mamba-2 HPO (16.20 ± 2.01), GRU baseline (13.58 ± 1.02), Mamba-1 (12.59), Perceiver IO (1.97), Transformer (1.01). GRU 250M (22.45) — за счёт пятикратного увеличения шагов.

GRU устойчив к lr от $1e-4$ до $4e-4$. Mamba-2 — при отклонении от оптимума награда падает на 20-30%. Perceiver IO чувствителен к размеру латентного массива. Transformer ограничен окном контекста — HPO бесполезна, пока window_size не увеличится.

3.4.2 Анализ потребления VRAM и скорости инференса

Помимо качества, важны VRAM и FPS. В табл. 6 — замеры core-модулей. Таблица 6 — Потребление ресурсов core-модулями: параметры, VRAM, FPS.

Архитектура	Параметры	VRAM (core)	FPS
GRU	0.8M	0.5 GB	24 000–56 000
Mamba-2	2.1M	1.2 GB	15 000–23 000
Transformer	1.0M	0.8 GB	29 000
Perceiver IO	2.5M	1.5 GB	8 000

GRU — самый лёгкий. Mamba-2 вдвое тяжелее из-за ssm_state (131K элементов). Perceiver IO — самый тяжёлый: 1.5 GB на core из-за латентного массива и двух механизмов внимания.

По FPS GRU лидирует. Mamba-2 медленнее из-за Triton-ядер SSM и копирования состояния между очередями, но 15-23K fps достаточно для on-policy. Perceiver IO — 8K, кросс-внимание бутылочное горлышко.

Полная модель (энкодер + core + декодер + буферы) — от 5 до 9 GB. Энкодер ViZDoom занимает основной объём, так что разница между архитектурами не так критична.

3.4.3 Обсуждение применимости архитектур

GRU — универсальный выбор по умолчанию. Работает без настройки, даёт 13-14 за 50M, до 22 за 250M. FPS 24-56K. Подходит, когда бюджет ограничен и HPO нет.

GRU + оптимизированные HP — лучший вариант при наличии времени на HPO. +31% к baseline, архитектура не меняется. Меняются только гиперпараметры оптимизатора: AdamW вместо стандартного, lr $4.05e-4$ вместо $1e-4$, weight_decay $1.96e-3$.

Mamba-2 — альтернатива для длинных последовательностей. На 11% ниже GRU при 50M, но $O(L)$ вместо $O(L^2)$. Без HPO падает ниже GRU baseline.

Transformer — не рекомендуется для визуального RL с окном менее 256 шагов.

Perceiver IO — с 32 латентами не даёт приемлемого качества. 1.5 GB core, 8K FPS — и результат 1.97.

ЗАКЛЮЧЕНИЕ

В работе рассматривался вопрос выбора архитектуры памяти для RL-агента. Сравнивались GRU, Mamba-2, Transformer и Perceiver IO — для этого на базе ViZDoom и Sample Factory был собран стенд, в котором модуль памяти заменяется через ModelCore без модификации энкодера и декодера. Каждая архитектура потребовала своей реализации: для Mamba-2 потребовалась реализация сериализации `conv_state + ssm_state` в `rnn_states` и `gradient checkpointing`, TransformerCore работает со скользящим окном и каузальной маской, PerceiverCore — с латентным массивом и кросс-вниманием. Гиперпараметры Mamba-2 подбирались байесовской оптимизацией — 85 trials, из которых 54% были прерваны по причине OOM. Для Transformer провели 15 trials. Стандартный прогон — 50M шагов на архитектуру, 3–4 seed для ключевых конфигураций, а также GRU 250M для оценки потолка.

GRU с гиперпараметрами Mamba-2 (AdamW, $\text{lr}=4\text{e-}4$) обеспечивает 17.86 ± 2.11 против 13.58 ± 1.02 у стандартного GRU — прирост +31% только за счёт настройки. Сама Mamba-2 (HPO best) достигает 16.20 ± 2.01 , то есть на 11% ниже, но при линейной сложности $O(L)$ против квадратичной у трансформера. GRU 250M достиг 22.45, после 150M шагов роста нет. Mamba-1 (12.59), Transformer (1.01) и Perceiver IO (1.97) существенно уступают — первые две архитектуры не дают приемлемого качества в данной постановке.

GRU остаётся предпочтительным выбором для on-policy визуального RL благодаря низким вычислительным требованиям, устойчивости к неточной настройке гиперпараметров и предсказуемому выходу на плато после 150M шагов. Оптимизация гиперпараметров даёт прирост, сопоставимый со сменой архитектуры — этот результат значим сам по себе. Mamba-2 — перспективная альтернатива: качество, близкое к GRU, при линейной сложности инференса, однако без тщательной настройки HP её потенциал не раскрывается.

Из сравнения видно, что выбор архитектуры определяется доступными вычислительными ресурсами. При ограниченном бюджете (до

50M шагов, без HPO) GRU со стандартными настройками Sample Factory выдаёт 13–14 и не требует настройки. Если время на HPO есть, тот же GRU с AdamW и lr $4e-4$ поднимается до 17–20 — лучший показатель среди всех архитектур при 50M. Mamba-2 интересна там, где важна скорость инференса на длинных последовательностях (сотни шагов): её $O(L)$ даёт 16–19, но без HPO она уступает стандартному GRU. Transformer и Perceiver IO для doom_benchmark не подходят: они не достигают приемлемого качества при фиксированном размере наблюдения и без кардинального увеличения окна контекста.

Дальнейшие направления включают увеличение окна контекста Transformer до 256+ шагов (чтобы проверить, решит ли это проблему), гибридные VLA-модели вроде CombatVLA [16], где Mamba-2 может выступать в роли backbone, и ablation studies для оценки вклада gradient checkpointing и размерности состояния SSM в общую производительность.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Kempka M. и др. ViZDoom: A Doom-based AI Research Platform for Visual Reinforcement Learning [Электронный ресурс]. 2016. URL: <https://arxiv.org/abs/1605.02097>.
2. Magne L. и др. NitroGen: An Open Foundation Model for Generalist Gaming Agents [Электронный ресурс]. 2025. URL: <https://nitrogen.minedojo.org/assets/documents/nitrogen.pdf>.
3. Bolton A. и др. SIMA 2: A Generalist Embodied Agent for Virtual Worlds [Электронный ресурс]. 2025. URL: <https://arxiv.org/abs/2512.04797>.
4. Cho K. и др. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation // arXiv preprint arXiv:1406.1078. 2014.
5. Vaswani A. и др. Attention Is All You Need // Advances in Neural Information Processing Systems 30 (NeurIPS). 2017.
6. Dao T., Gu A. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality [Электронный ресурс]. 2024. URL: <https://arxiv.org/abs/2405.21060>.
7. Jaegle A. и др. Perceiver IO: A General Architecture for Structured Inputs & Outputs [Электронный ресурс]. 2022. URL: <https://arxiv.org/abs/2107.14795>.
8. Smirnov I., Gu S. RLBenchNet: The Right Network for the Right Reinforcement Learning Task [Электронный ресурс]. 2025. URL: <https://arxiv.org/abs/2505.15040>.
9. Gu A., Dao T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces [Электронный ресурс]. 2023. URL: <https://arxiv.org/abs/2312.00752>.
10. Schulman J. и др. Proximal Policy Optimization Algorithms [Электронный ресурс]. 2017. URL: <https://arxiv.org/abs/1707.06347>.
11. Petrenko A. и др. Sample Factory: Egocentric 3D Control from Pixels at 100000 FPS with Asynchronous Reinforcement Learning [Электронный ресурс]. 2020. URL: <https://arxiv.org/abs/2006.11751>.
12. Hafner D. Deep Reinforcement Learning From Raw Pixels in Doom [Электронный ресурс]. 2016. URL: <https://arxiv.org/abs/1610.02164>.

13. Mnih V. и др. Human-level Control through Deep Reinforcement Learning // Nature. 2015. Т. 518, № 7540. Сс. 529–533.
14. Hochreiter S., Schmidhuber J. Long Short-Term Memory // Neural Computation. 1997. Т. 9, № 8. Сс. 1735–1780.
15. Xiao C. и др. Spatial-Mamba: Effective Visual State Space Models via Structure-aware State Fusion [Электронный ресурс]. 2024. URL: <https://arxiv.org/abs/2410.15091>.
16. Chen P. и др. CombatVLA: An Efficient Vision-Language-Action Model for Combat Tasks in 3D Action Role-Playing Games [Электронный ресурс]. 2025. URL: <https://arxiv.org/abs/2503.09527>.

ПРИЛОЖЕНИЕ А

Листинги кода

ПРИЛОЖЕНИЕ А.1

Mamba2StateEncoder

```
class Mamba2StateEncoder:
    """Encodes/decodes Mamba-2 internal state (conv_state +
    ssm_state)
    to/from a flat tensor that fits in rnn_states."""

    def __init__(self, d_ssm: int, d_conv_dim: int, nheads: int,
                  headdim: int, d_state: int, d_conv: int):
        self.conv_state_size = d_conv_dim * d_conv
        self.ssm_state_size = nheads * headdim * d_state
        self.total_size = self.conv_state_size + self.ssm_state_size

    def encode(self, conv_state, ssm_state) -> torch.Tensor:
        """Flatten conv_state (batch, d_conv_dim, d_conv)
        and ssm_state (batch, nheads, headdim, d_state)
        into one tensor (batch, total_size)."""
        conv_flat = conv_state.reshape(conv_state.shape[0], -1)
        ssm_flat = ssm_state.reshape(ssm_state.shape[0], -1)
        return torch.cat([conv_flat, ssm_flat], dim=-1)

    def decode(self, flat_state, device=None, dtype=None):
        """Split flat tensor back into conv_state and ssm_state."""
        batch = flat_state.shape[0]
        conv_flat = flat_state[:, :self.conv_state_size]
        ssm_flat = flat_state[:, self.conv_state_size:]
        conv_state = conv_flat.reshape(batch, self.d_conv_dim,
self.d_conv)
        ssm_state = ssm_flat.reshape(batch, self.nheads,
self.headdim, self.d_state)
        return conv_state, ssm_state
```

Листинг B.1 — Mamba2StateEncoder

ПРИЛОЖЕНИЕ А.2

Mamba2Core.forward

```
class Mamba2Core(ModelCore):
    def forward(self, head_output, rnn_states):
        is_seq = not torch.is_tensor(head_output)

        if is_seq:
            # Training: PackedSequence -> (T, B, D)
            x_data = _unpack_packed_sequence_2d(head_output)
        else:
            # Inference: (B, D) -> (1, B, D)
            x_data = head_output.unsqueeze(0)

        # Mamba expects (B, T, D)
        x_data = x_data.permute(1, 0, 2)
        x_data = self.input_proj(x_data)

        if is_seq:
            # Training: process full sequence
            if getattr(self.cfg, 'gradient_checkpointing', True):
                x_data = self._forward_with_checkpointing(x_data)
            else:
                x_data = self.mamba_layers(x_data)
        else:
            # Inference: decode state, process one step, encode back
            batch_size = x_data.shape[0]
            inference_params =
self._create_inference_params(batch_size)
            self._load_states_from_rnn(inference_params, rnn_states)
            inference_params.seqlen_offset = 1

            for norm, wrapped in zip(self.mamba_norms,
self.mamba_wrapped):
                x_data = wrapped(norm(x_data),
                                inference_params=inference_params)

            new_rnn_states =
```

```

self._get_inference_states(inference_params)

x_data = x_data.permute(1, 0, 2) # back to (T, B, D)

if is_seq:
    x = _pack_to_2d_sequence(x_data, head_output)
    new_rnn_states = rnn_states
else:
    x = x_data.squeeze(0)

return x, new_rnn_states

```

Листинг B.2 — Mamba2Core.forward

ПРИЛОЖЕНИЕ A.3

Gradient checkpointing

```

def _forward_with_checkpointing(self, x: torch.Tensor) ->
torch.Tensor:
    from torch.utils.checkpoint import checkpoint

    def segment_forward(x, norm, mamba_layer):
        return mamba_layer(norm(x))

    output = x
    for norm, wrapped in zip(self.mamba_norms, self.mamba_wrapped):
        output = checkpoint(
            segment_forward,
            output, norm, wrapped,
            use_reentrant=False,
        )
    return output

```

Листинг B.3 — Gradient checkpointing

ПРИЛОЖЕНИЕ A.4

Регистрация Mamba-2 в sample-factory

```

class Mamba2Factory:
    __slots__ = ('_num_layers',)
    def __init__(self, num_layers):
        self._num_layers = num_layers

```

```

def __call__(self, cfg, core_input_size):
    if cfg.use_rnn and cfg.rnn_type == 'mamba2':
        cfg.rnn_num_layers = self._num_layers
        return Mamba2Core(cfg, core_input_size)
    return default_make_core_func(cfg, core_input_size)

def register_mamba2(cfg=None):
    num_layers = cfg.rnn_num_layers if cfg is not None else 1
    factory = Mamba2Factory(num_layers)
    global_model_factory().register_model_core_factory(factory)

```

Листинг В.4 — Регистрация Mamba-2 в sample-factory

ПРИЛОЖЕНИЕ А.5

TransformerCore

```

class TransformerCore(ModelCore):
    def __init__(self, cfg, input_size):
        self.d_model = getattr(cfg, 'transformer_d_model',
cfg.rnn_size)
        self.nhead = getattr(cfg, 'transformer_nhead', 8)
        self.num_layers = getattr(cfg, 'transformer_num_layers',
cfg.rnn_num_layers)
        self.window_size = getattr(cfg, 'transformer_window_size',
64)

        self.dim_feedforward = getattr(cfg,
'transformer_dim_feedforward', self.d_model * 4)

        self.state_encoder = TransformerStateEncoder(self.d_model,
self.window_size)
        self.layers = nn.ModuleList()
        for _ in range(self.num_layers):
            self.layers.append(nn.TransformerEncoderLayer(
                d_model=self.d_model, nhead=self.nhead,
                dim_feedforward=self.dim_feedforward,
                dropout=0.1, activation='gelu',
                batch_first=True, norm_first=True,
            ))

```

```

self.input_proj = nn.Linear(input_size, self.d_model)
self.pos_embedding = nn.Parameter(
    torch.randn(1, self.window_size, self.d_model) * 0.02)
self.core_output_size = self.d_model

def forward(self, head_output, rnn_states):
    # unpack, project -> (B, T, D)
    x_data = _unpack_packed_sequence_2d(head_output).permute(1,
0, 2)
    x_data = self.input_proj(x_data)
    B, T, D = x_data.shape

    if T > 1: # Training: causal mask
        x_data = x_data + self.pos_embedding[:, :T, :]
        mask = _causal_mask(T, x_data.device)
        for norm, layer in zip(self.norms, self.layers):
            x_data = layer(x_data, src_mask=mask)
        out = x_data[:, -1:, :]
        new_rnn_states = rnn_states
    else: # Inference: sliding window
        buffer = self.state_encoder.decode(rnn_states, B)
        buffer = torch.roll(buffer, shifts=-1, dims=1)
        buffer[:, -1:, :] = x_data + self.pos_embedding[:,
-1:, :]

        mask = _causal_mask(self.window_size, x_data.device)
        for norm, layer in zip(self.norms, self.layers):
            buffer = layer(buffer, src_mask=mask)
        out = buffer[:, -1:, :]
        new_rnn_states =
self.state_encoder.encode(buffer.detach())

    out = out.permute(1, 0, 2)
    x = _pack_to_2d_sequence(out.expand(T, -1, -1).contiguous(),
head_output)
    return x, new_rnn_states

```

Листинг B.5 — TransformerCore

ПРИЛОЖЕНИЕ А.6

PerceiverCore

```
class PerceiverCore(ModelCore):
    def __init__(self, cfg, input_size):
        self.num_latents = getattr(cfg, 'perceiver_num_latents', 32)
        self.d_latents = getattr(cfg, 'perceiver_d_latents', 512)
        self.num_blocks = getattr(cfg, 'perceiver_num_blocks', 2)

        self.state_encoder = PerceiverStateEncoder(self.num_latents,
self.d_latents)
        self.input_proj = nn.Linear(input_size, self.d_model)
        self.latent = nn.Parameter(torch.randn(1, self.num_latents,
self.d_latents) * 0.02)
        self.pos_encoding = nn.Parameter(torch.randn(1, 1024,
self.d_model) * 0.02)

        self.cross_blocks = nn.ModuleList([
            CrossAttentionBlock(self.d_latents, self.d_model,
num_heads=8)
            for _ in range(self.num_blocks)
        ])
        self.self_blocks = nn.ModuleList([
            SelfAttentionBlock(self.d_latents, num_heads=8)
            for _ in range(self.num_blocks)
        ])
        self.output_proj = nn.Sequential(
            nn.LayerNorm(self.d_latents), nn.Linear(self.d_latents,
self.d_model))
        self.core_output_size = self.d_model

    def forward(self, head_output, rnn_states):
        x_data = _unpack_packed_sequence_2d(head_output).permute(1,
0, 2)
        x_data = self.input_proj(x_data)
        B, T, D = x_data.shape

        if T > 1: # Training
```

```

        x_data = x_data + self.pos_encoding[:, :T, :]
        latent = self.latent.expand(B, -1, -1)
        for cross, self_attn in zip(self.cross_blocks,
self.self_blocks):
            latent = cross(latent, x_data)
            latent = self_attn(latent)
        out = self.output_proj(latent.mean(dim=1)).unsqueeze(1)
        new_rnn_states = rnn_states
    else: # Inference
        latent = self.latent.expand(B, -1, -1).detach()
        if rnn_states.norm().item() > 1e-6:
            latent = self.state_encoder.decode(rnn_states, B)
        x_data = x_data + self.pos_encoding[:, :1, :]
        latent = self.cross_blocks[0](latent, x_data)
        latent = self.self_blocks[0](latent)
        out = self.output_proj(latent.mean(dim=1)).unsqueeze(1)
        new_rnn_states =
self.state_encoder.encode(latent.detach())

    out = out.permute(1, 0, 2)
    x = _pack_to_2d_sequence(out.expand(T, -1, -1).contiguous(),
head_output)
    return x, new_rnn_states

```

Листинг B.6 — PerceiverCore

ПРИЛОЖЕНИЕ Б

Графики обучения

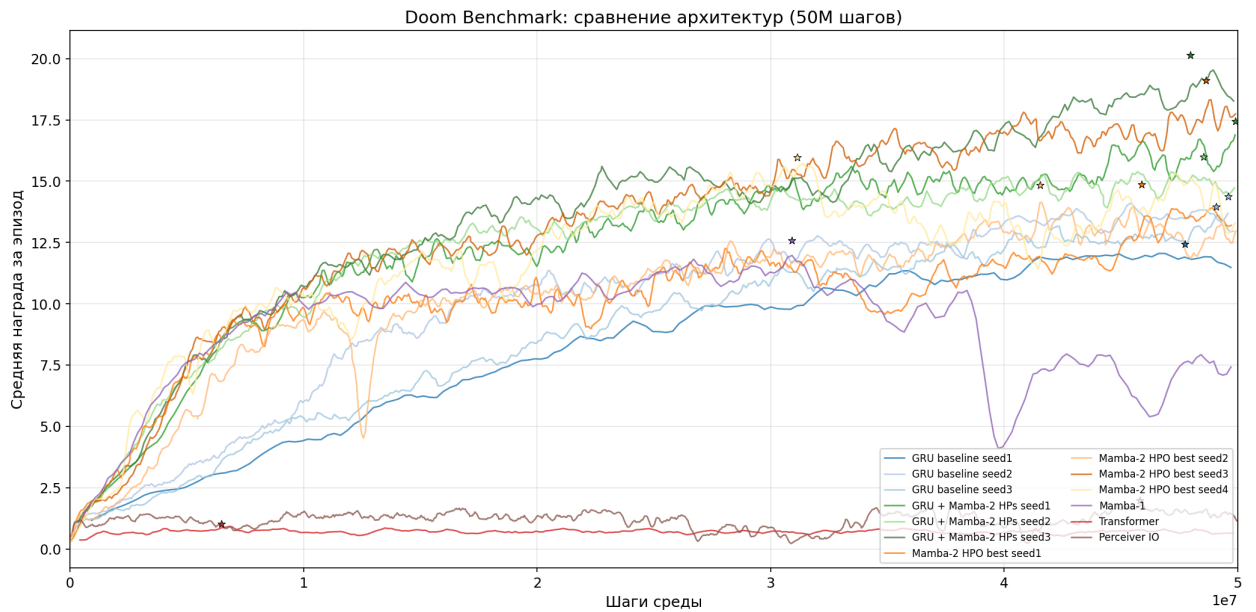


Рисунок Б.1 — Рис. А.1 — Кривые обучения всех архитектур на 50М шагов.

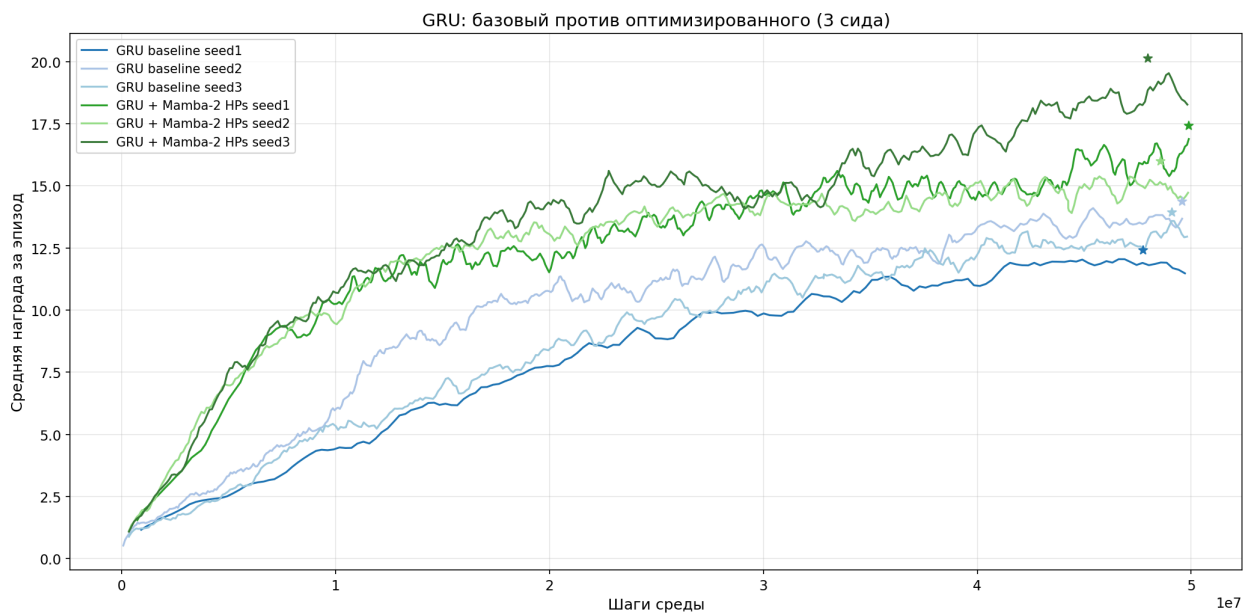


Рисунок Б.2 — Рис. А.2 — GRU baseline против GRU с гиперпараметрами Mamba-2 (3 seed)

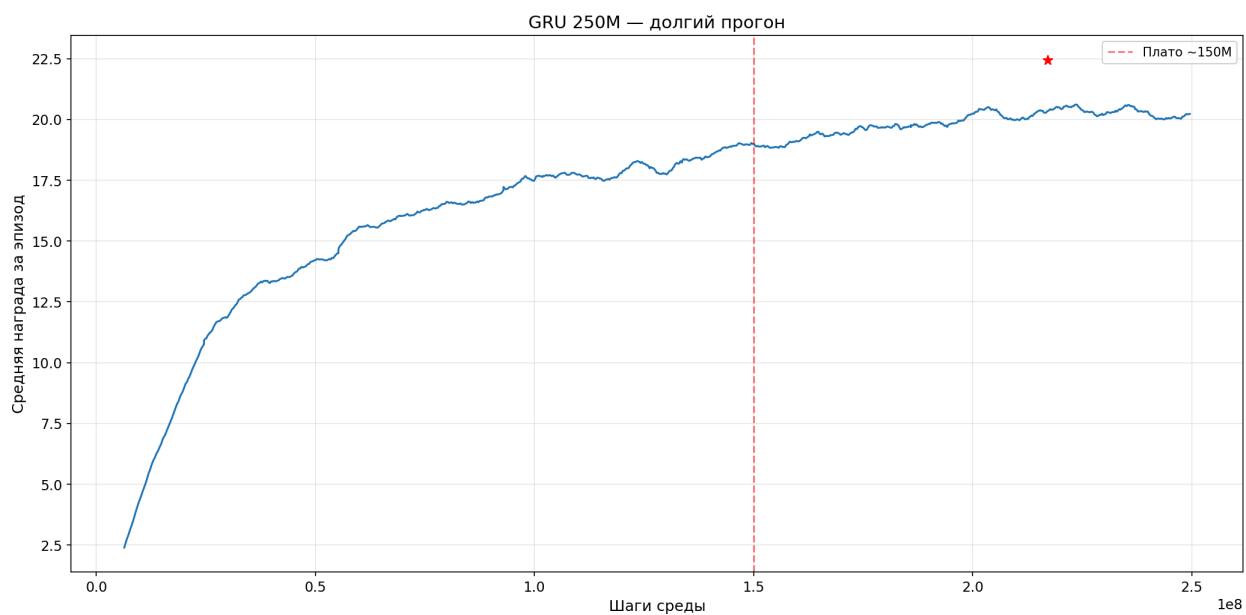


Рисунок Б.3 — Рис. А.3 — GRU 250M. Плато после 150M, максимальная награда 22.45

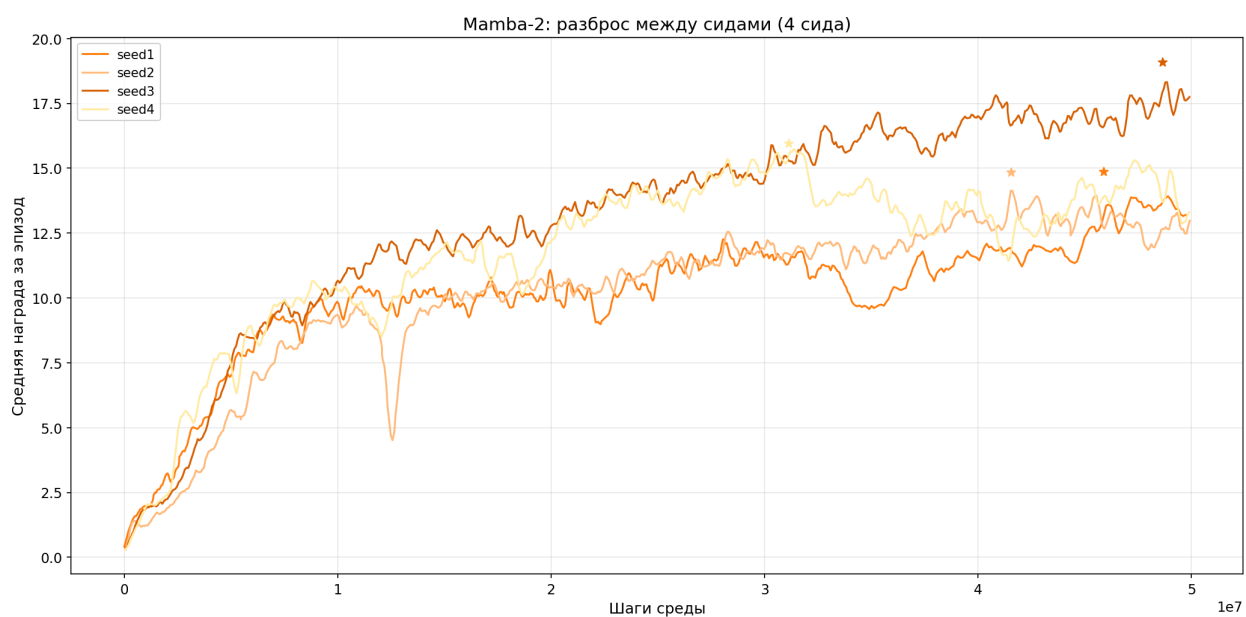


Рисунок Б.4 — Рис. А.4 — Mamba-2 HPO: разброс между 4 seed

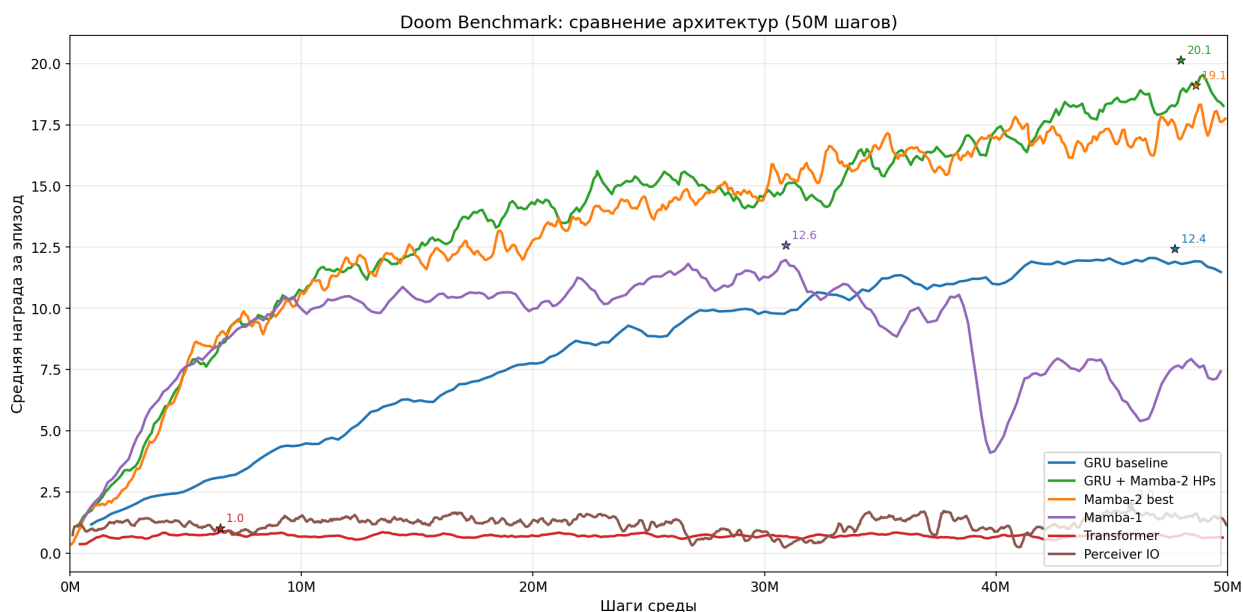


Рисунок Б.5 — Рис. А.5 — Пилотные эксперименты GRU и Mamba-2 на 10M шагов

Результаты полного обучения:

Таблица Б.1 — Таблица А.1 — Сводные результаты экспериментов. ^аПосле НРО. ^бПо последним 20% итераций.

Архитектура	Seed	Mean	Std	Max	FPS	VRAM
GRU baseline	3	13.58	1.02	14.37	24–56K	6 GB
GRU + Opt.HP	3	17.86	2.11	20.15	17–25K	6 GB
Mamba-2 HPO	4	16.20	2.01	19.10	15–23K	7 GB
Mamba-1	1	12.59	—	12.59	37K	6 GB
Transformer	1	1.01	—	2.03 ^а	29K	6 GB
Perceiver IO	1	1.97	—	1.97	8K	8 GB
GRU 250M	1	20.28 ^б	0.66 ^б	22.45	56K	6 GB

ПРИЛОЖЕНИЕ В

Конфигурации экспериментов

ПРИЛОЖЕНИЕ В.1

Базовая конфигурация (APPO)

Таблица В.1 — Базовая конфигурация APPO

Продолжение таблицы В.1

Параметр	Значение
algorithm	APPO
env	doom_benchmark
num_workers	8
num_envs_per_worker	8
batch_size	4096
rollout	64
recurrence	32
hidden_size	512
rnn_num_layers	1
learning_rate	1e-4 (GRU) / 4.05e-4 (Mamba-2)
optimizer	AdamW
weight_decay	0 (GRU) / 1.96e-3 (Mamba-2)
gamma	0.99
gae_lambda	0.95
value_loss_coeff	1.0
exploration_loss_coeff	0.002 (Mamba-2)
grad_norm	4.0
num_epochs	1
num_minibatches	1

ПРИЛОЖЕНИЕ В.2

Конфигурация Mamba-2 (HPO best trial 62)

Таблица В.2 — Конфигурация Mamba-2, trial 62

Параметр	Значение
mamba_d_model	512
mamba_d_state	128
mamba_headdim	128
mamba_expand	1
mamba_d_conv	4
mamba_ngroups	1
gradient_checkpointing	True
rnn_type	mamba2
learning_rate	4.05e-4
weight_decay	1.96e-3
optimizer	AdamW
exploration_loss_coeff	0.002

ПРИЛОЖЕНИЕ В.3

Конфигурация TransformerCore

Таблица В.3 — Конфигурация TransformerCore

Параметр	Значение
transformer_d_model	512
transformer_nhead	8
transformer_num_layers	2
transformer_window_size	64
transformer_dim_feedforward	2048
transformer_dropout	0.1
rnn_type	transformer

ПРИЛОЖЕНИЕ В.4

Конфигурация PerceiverCore

Таблица В.4 — Конфигурация PerceiverCore

Параметр	Значение
perceiver_num_latents	32
perceiver_d_latents	512
perceiver_num_blocks	2
perceiver_num_heads	8
perceiver_dropout	0.1
rnn_type	perceiver

ПРИЛОЖЕНИЕ В.5

Версии программного обеспечения

Таблица В.5 — Версии программного обеспечения экспериментального стенда

Компонент	Версия
PyTorch	2.11.0+cu130
Sample Factory	2.1.1
ViZDoom	1.3.0
CUDA	13.0
cuDNN	9.1.9
Python	3.13.12

ПРИЛОЖЕНИЕ В.6

Потребление видеопамяти (core-only)

В табл. В.6 сведены результаты замеров core-модулей (энкодер и декодер в расчёт не брались). Число параметров и потребление видеопамяти не всегда коррелируют напрямую: Mamba-2 с 0.9M параметров даёт 0.40 GB, а Perceiver IO, у которого параметров больше в 16 раз, — всего 0.42 GB. SSM-сканирование вынуждено хранить conv_state и ssm_state на каждом шаге, а латентный массив Perceiver IO фиксирован (32×512) и не растёт с длиной последовательности. Полная модель даёт 5–9 GB, основная доля уходит на свёрточный энкодер с его 4096 кадрами.

Таблица В.6 — Потребление видеопамяти core-only (без учёта энкодера и декодера)

Архитектура	Параметры (core)	VRAM (core)
GRU	1.6M	0.07 GB
Mamba-2	0.9M	0.40 GB
Mamba-1	1.7M	0.15 GB
Transformer	3.2M	0.20 GB
Perceiver IO	14.5M	0.42 GB